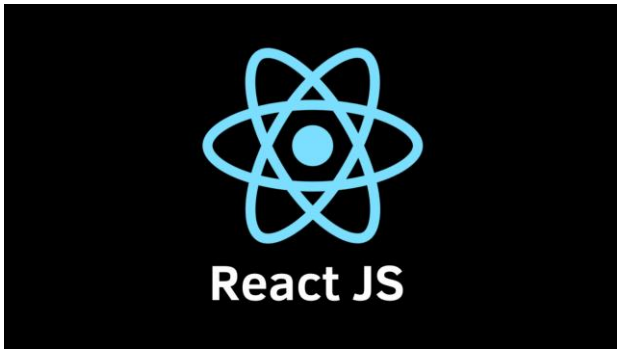
Express JS



TypeScript

Satya Kaveti

1. NODE.JS **4**

ARCHITECTURE MODEL 5
NODE.JS CONSOLE - REPL 7
NODE.JS BASICS 8
NODE.JS MODULES 9
NPM - NODE PACKAGE MANAGER 11
NODEJS EXAMPLES – MODULE BY MODULE 13
NODE.JS – FRAMEWORKS 17
REF. 18

2. EXPRESS.JS **19**

EXPRESS.JS WEB APPLICATION 20
EXPRESS.JS – DATABASE ACCESS 24
NODE.JS - TEMPLATE ENGINES 26

3. REACT JS **28**

REACT - INTRODUCTION 28
REACT - CREATE REACT APP 30
REACT - JSX - JAVASCRIPT + XML 32
REACT – COMPONENTS 34
REACT – DATA HANDLING 37
REACT - LIFECYCLE METHODS 38
REACT – ROUTING 38
REF. 40

4. TYPE SCRIPT **41**

INTRODUCTION TO TYPESCRIPT 41
FEATURES OF TYPESCRIPT 41
DATA TYPES 42
FUNCTIONS 44
STRINGS 46
ARRAYS 46
INTERFACES 46
CLASSES 47
OBJECTS 48

5. ANGULAR **49**

INSTALL ANGULAR 49
TYPESCRIPT – DATATYPES 50
CREATE ANGULAR APPLICATION 52
BUILDING BLOCKS OF ANGULAR 54
ANGULAR COMPONENT 54

6. APPLICATIONS FOR HANDS-ON **59**

#1 - REACT + SPRINGBOOT 59
#2 - ANGULAR + SPRINGBOOT 69
SPRINGBOOT + THYMELEAF 78

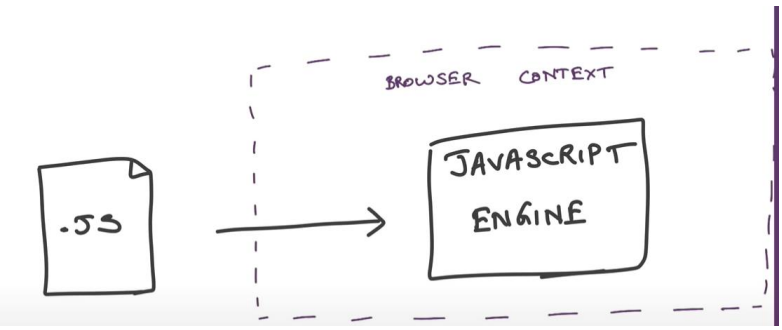
DOCKERIZE SPRING MVC APPLICATION 78
DEPLOYING SPRING BOOT APPLICATIONS TO HEROKU 78
SPRINGBOOT – MICRO SERVICES 78

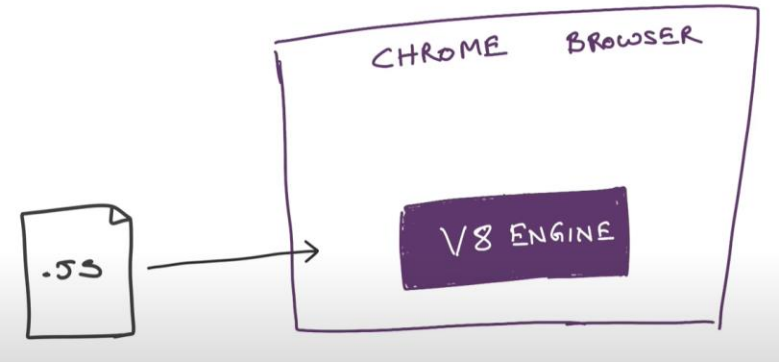
MYTRAILS CTMS SAMPLE PROJECT 79

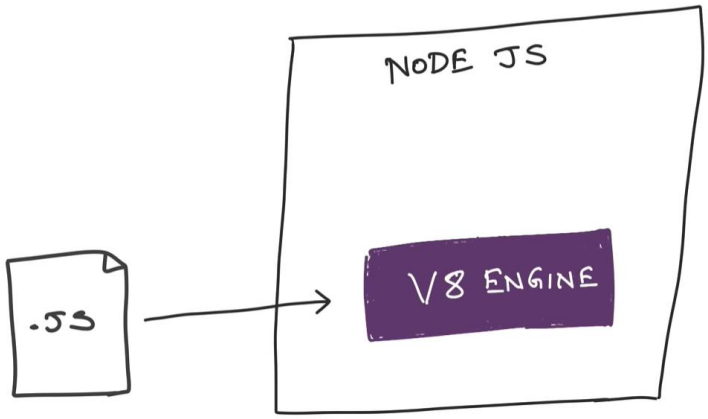
PHOENIX CTMS 79
LIBRECLINICA 79

1. Node.js

Node.js is an open-source server-side runtime environment built on Chrome's V8 JavaScript engine. It provides an event driven, non-blocking (asynchronous) I/O and cross-platform runtime environment for building highly scalable server-side applications using JavaScript.





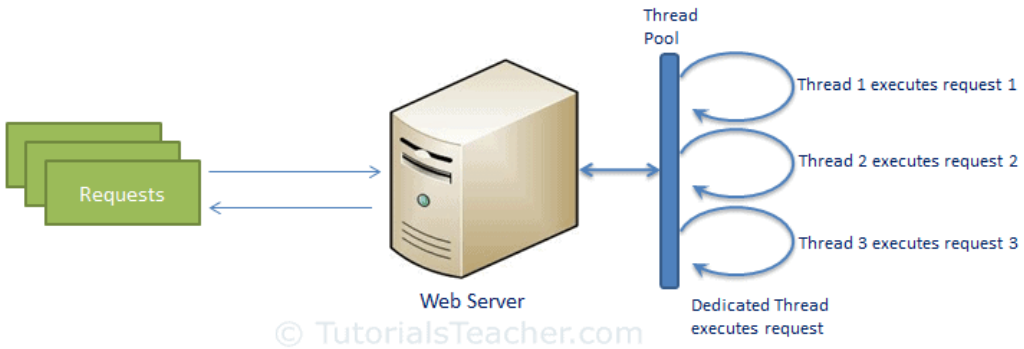


Node.js can be used to build different types of applications such as **command line application, web application, real-time chat application, REST API server** etc.

Architecture model

Normal Sever Request Processing

In Normal web server model, each request is handled by a dedicated thread from the thread pool. If no thread is available in the thread pool, then the request waits till the next available thread.

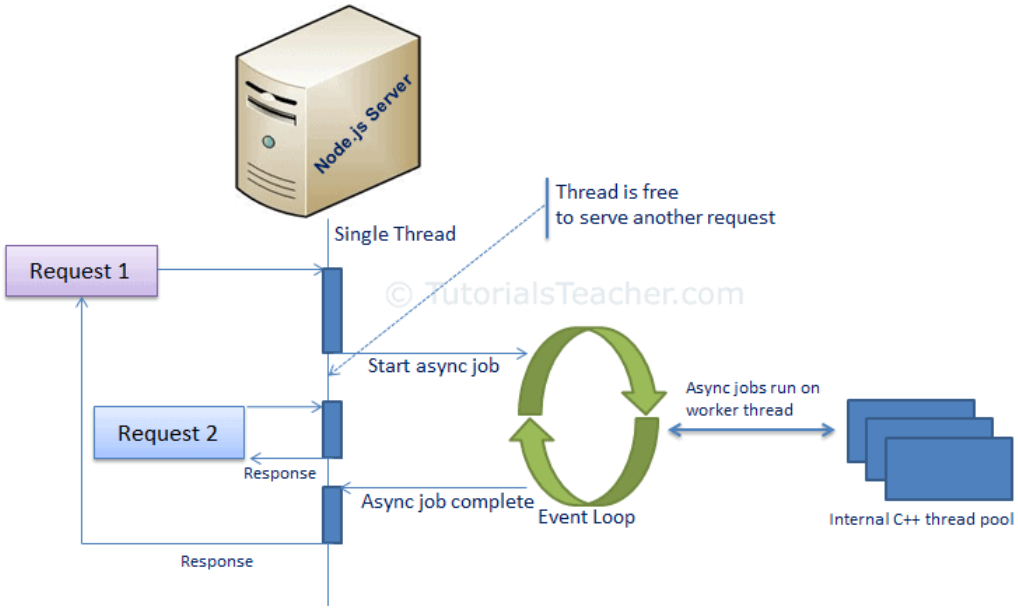


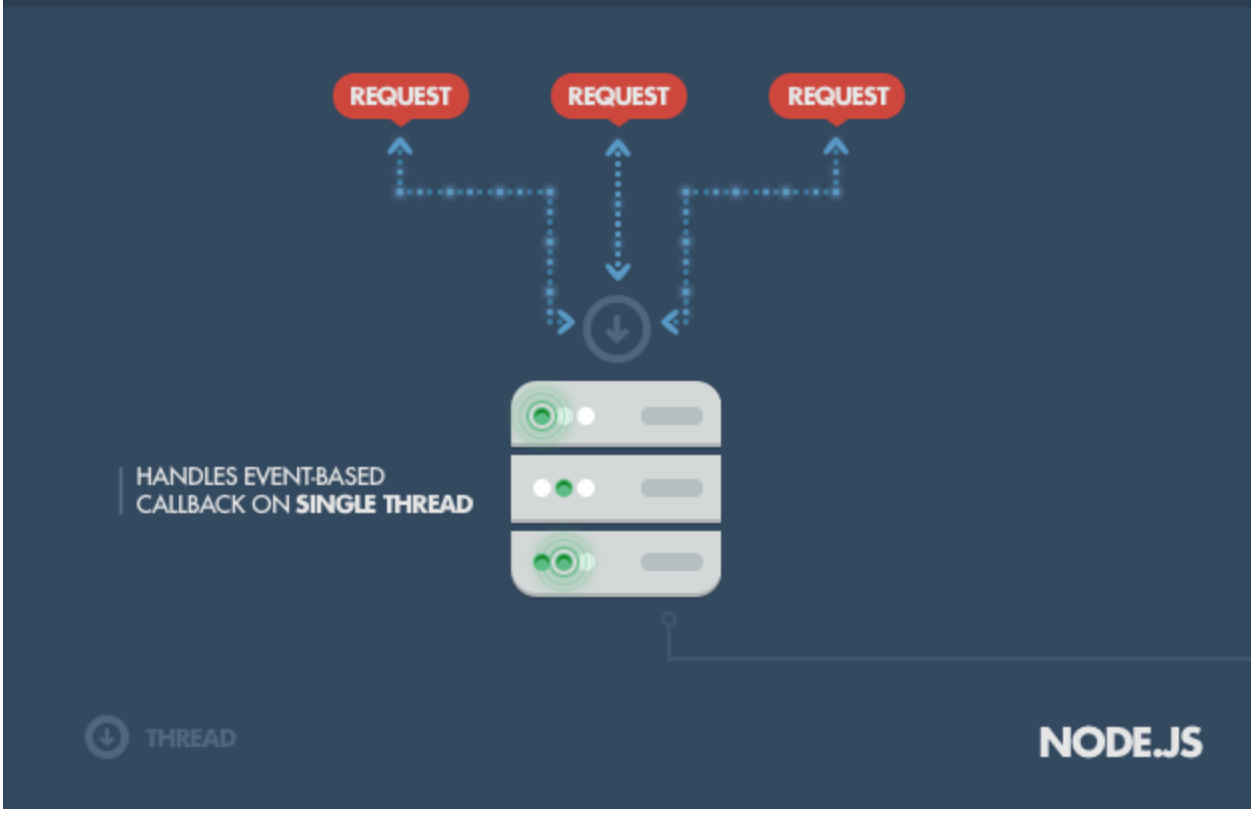
Nodejs Request Processing

Node.js runs in a **single process** and the application code runs in a **single thread**. All the user requests to your web application will be handled by a single thread and all the I/O work or long running job is performed asynchronously for a particular request.

So, this single thread doesn't have to wait for the request to complete and is free to handle the next request. When asynchronous I/O work completes then it processes the request further and sends the response.

An event loop is constantly watching for the events to be raised for an asynchronous job and executing callback function when the job completes.





Node.js is not fit for an application which performs CPU-intensive operations like image processing or other heavy computation work because it takes time to process a request and thereby blocks the single thread.

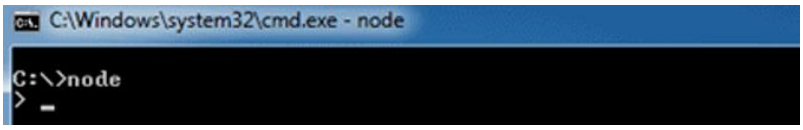
Satya Kaveti

Java	vs	NodeJS
December, 1999	The first release	May, 2009
Lower execution speed	Performance	High speed due to non-blocking processing
Both vertical and horizontal	Scalability	Horizontal
Ready to use security solutions	Security	Customized security strategy
Lengthy and relatively slower	Development process	Fast and less complicated
106,353	Number of enterprises applying	49,383
TripAdvisor, GameDuell, Rezdy	Use cases	Uber, Netflix, eBay, Reddit
Oracle, the Eclipse Foundation	Supported by	Community and the Node.js Foundation
$101,142	Developer's average annual salary	$104,964

Node.js Console - REPL

Node.js comes with virtual environment called REPL (aka Node shell). REPL stands for `Read-Eval-Print-Loop`. It is a quick and easy way to test simple Node.js/JavaScript code.

To Start REPL, just type `node`



Satya Kaveti

```
koushikkothagal — node — 80×24
koushikkothagal@Koushiks-MacBook-Pro-2 ~ % node -v
v13.2.0
koushikkothagal@Koushiks-MacBook-Pro-2 ~ % node
Welcome to Node.js v13.2.0.
Type ".help" for more information.
> 1 + 1
2
> var a;
undefined
> a = 10;
10
> a
10
> a * 2
20
> var b = a * 3;
undefined
> b
30
>
```

To execute JavaScript file from cmdline, use `node <.js-filename>`

```
//index.js
console.log("Hello, Node.js");
```

```
C:\NodeApp>node index.js
Hello, Node.js
```

Node.js Basics

Variables

to create a variable in Node.js, you need to make use of a reserved keyword `var`. You do not have to assign a data type, as **the compiler will automatically pick it**.

Datatypes

Node.js has various datatypes, categorized into Primitive and Non-Primitive datatypes.

Primitive Data Types are:

1. String
2. Number
3. Boolean
4. Null
5. Undefined

```
var str = "Satya";
var num = 10;
```

All the Datatypes are taken care by compiler – based on data we are assigning; we don't need to define datatype manually.

Non-Primitive Data Types are:

- Object
- Date
- Array

```
var obj = {
    authorName: 'Ryan Dahl',
    language: 'Node.js'
}
```

Buffer

Node.js includes an additional data type called Buffer (not available in browser's JavaScript). Buffer is mainly used to store binary data, while reading from a file or receiving packets over the network.

Functions

A function can have attributes and properties also. It can be treated like a class in JavaScript.

```
function Display(x) {
    console.log(x);
}

Display(100);
```

Global Scope

In a browser, global scope is the **window** object.

```
window.location = "http://www.yoururl.com";
```

In Node.js, **global** object represents the **global scope**.

Node.js Modules

Module in Node.js is a simple or complex functionality organized in single or multiple JavaScript files which can be reused throughout the Node.js application.

Each module in Node.js has its own context, so it cannot interfere with other modules or pollute global scope. Also, each module can be placed in a separate .js file under a separate folder.

Node.js includes three types of modules:

1. **Core Modules**
2. **Local Modules**
3. **Third Party Modules**

1.Core Modules

Node.js is a lightweight framework. The core modules include bare minimum functionalities of Node.js. These core modules are compiled into its binary distribution and load automatically when Node.js process starts. However, you **need to import the core module** first in order to use it in your application.

The following table lists some of the important core modules in Node.js.

Core Module	Description
`http`	http module includes classes, methods and events to create Node.js http server.
`url`	url module includes methods for URL resolution and parsing.
`querystring`	querystring module includes methods to deal with query string.
`path`	path module includes methods to deal with file paths.
`fs`	fs module includes classes, methods, and events to work with file I/O.
`util`	util module includes utility functions useful for programmers.

In order to use Node.js core or NPM modules, you first need to import it using `require()` function as below.

```
var module = require('module_name');
```

As per above syntax, specify the module name in the `require()` function. The `require()` function will return an object, function, property or any other JavaScript type, depending on what the specified module returns.

The following example demonstrates how to use Node.js `http` module to create a web server.

```
var http = require('http');

var server = http.createServer(function(req, res){
  //write code here
});
server.listen(5000);
```

In the above example, require() function returns an object because http module returns its functionality as an object, you can then use its properties and methods using dot notation e.g. `http.createServer().`

2.Local Module

Local modules are modules created locally in your Node.js application. These modules include different functionalities of your application in separate files and folders. You can also package it and distribute it via NPM, so that Node.js community can use it.

For example, if you need to connect to MongoDB and fetch data then you can create a module for it, which can be reused in your application.

Writing Simple Module

Let's write simple logging module which logs the **information, warning** or **error** to the console.

log.js

```
var log = {
            info: function (info) {
                console.log('Info: ' + info);
            },
            warning:function (warning) {
                console.log('Warning: ' + warning);
            },
            error:function (error) {
                console.log('Error: ' + error);
            }
    };

module.exports = log
```

Loading Local Module – `app.js`

```
var myLogModule = require('./log.js');

myLogModule.info('Node.js started');
```

Run the above example using command prompt (in Windows) as shown below.

```
C:\> node app.js
Info: Node.js started
```

In the above example of logging module, we have created an object with three functions `- info(), warning() and error()`. At the end, we have assigned this object to `module.exports.` The module.exports in the above example exposes a log object as a module.

The `module.exports` is a special object which is included in every JS file in the Node.js application by default. Use **module.exports** or **exports** to expose a function, object or variable as a module in Node.js.

3.External Modules

You can use the external or 3rd party modules only by downloading them via **NPM**. These modules are generally developed by other developers and are free to use. Few of the best external modules are *express, react, gulp, mongoose, mocha etc*.

NPM - Node Package Manager

- Node Package Manager (NPM) is a command line tool that **installs, updates or uninstalls** Node.js packages in your application.
- It is also an **online repository** for open-source Node.js packages.
- The node community around the world creates useful modules and publishes them as packages in this repository.
- Official website: https://www.npmjs.com

NPM is included with Node.js installation. After you install Node.js, verify NPM installation by writing the following command in terminal or command prompt.

```
C:\NodeApp>npm -v
6.14.12
```

Install Package Locally

Use the following command to install any third party module in your local Node.js project folder.

```
C:\>npm install <package name>
```

For example, the following command will install ExpressJS into MyNodeProj folder.

```
C:\MyNodeProj> npm install express
```

All the modules installed using NPM are installed under **node_modules** folder. The above command will create ExpressJS folder under node_modules folder in the root folder of your project and install Express.js there.

Save package in package.json

Use `--save` at the end of the install command to add dependency entry into `package.json` of your application.

```
npm install express --save
```

Install Package Globally

NPM can also install packages globally so that all the node.js application on that computer can import and use the installed packages. NPM installs global packages into `/<User>/local/lib/node_modules` folder.

Apply `-g` in the install command to install package globally. For example, the following command will install ExpressJS globally.

```
C:\MyNodeProj> npm install -g express
```

```
## Update package
npm update express

## Uninstall package
npm uninstall express
```

Nodejs Examples – Module by Module

Before going to code, we must create a project to write a code. For that use `npm init &` Provide required input. It will create required files & package.json

```
C:\\Nodeapp> npm init
About to write to C:\NodeApp\package.json:
{
  "name": "nodeapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "node index.js"
  },
  "author": "Satya",
  "license": "ISC"
}
```

Node.js Server - HTTP module

Example 1: Create a Webserver

Node.js provides capabilities to create your own web server which will handle HTTP requests asynchronously.

```
// 1 - Import Node.js core module
var http = require('http');

// 2 - creating server
var server = http.createServer(function(req, res) {
  //Logic to handle requests
});

//3 - listen for incoming requests on port 5000
server.listen(5000);
console.log('Server Started ....')
```

```
c:\\NodeApp> node server.js
Server Started…
```

This is how you create a Node.js web server using simple steps. Now, let's see how to handle HTTP request and send response in Node.js web server.

Example 2 : Handle HTTP Request

The `http.createServer()` method includes request and response parameters which is supplied by Node.js.

- `request` object can be used to get information about the current HTTP request `e.g., url, request header, and data`.
- `response` object can be used to send a response for a current HTTP request.

```
// Import Node.js core module
var http = require('http');

//create web server
var server = http.createServer(function (req, res) {
    if (req.url == '/') { //check the URL of the current request

        // set response header
        res.writeHead(200, { 'Content-Type': 'text/html' });

        // set response content
        res.write('<html><body><p>This is home Page.</p></body></html>');
        res.end();

    }
    else if (req.url == "/student") {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.write('<html><body><p>This is student Page.</p></body></html>');
        res.end();
    }
    else if (req.url == "/admin") {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.write('<html><body><p>This is admin Page.</p></body></html>');
        res.end();
    }
    else
        res.end('Invalid Request!');

});

server.listen(5000); //6 - listen for any incoming requests

console.log('Node.js web server at port 5000 is running..')
```

In the above example,

- `req.url` is used to check the url of the current request and based on that it sends the response.
- To send a response, first it sets the response header using `writeHead()` method and then writes a string as a response body using `write()` method.
- Finally, Node.js web server sends the response using `end()` method.

Example 3 : Sending JSON Response

```
var http = require('http');

var server = http.createServer(function (req, res) {
    if (req.url == '/data') { //check the URL of the current request
            res.writeHead(200, { 'Content-Type': 'application/json' });
            res.write(JSON.stringify({ message: "Hello World"}));
            res.end();
    }
});
server.listen(5000);
```

Node.js File System – 'fs' module

Node.js includes **fs** module to access physical file system. The fs module is responsible for all the asynchronous or synchronous file I/O operations.

Read files

```
fs.readFile(fileName [,options], callback)
```

```
var fs = require('fs');

fs.readFile('a.txt',
    function (err, data) {
                    if (err){
                        throw err;
                    }
    console.log(data);
});
```

```
fs.readFileSync('dummyfile.txt', 'utf8');
```

```
var fs = require('fs');

var data = fs.readFileSync('a.txt', 'utf8');
console.log(data);
```

Write to files

```
fs.writeFile(filename, data[, options], callback)
```

```
var fs = require('fs');

fs.writeFile('test.txt', 'Hello World!', function (err) {
                        if (err)
        console.log(err);
                        else
        console.log('Write operation complete.');
});
```

```
fs.appendFile('test.txt', 'Hello World!', callback)
```

```
var fs = require('fs');

fs.appendFile('test.txt', 'Hello World!', function (err) {
                        if (err)
        console.log(err);
                        else
        console.log('Append operation complete.');
});
```

Delete files : `fs.unlink(path, callback);`

Rename files : `fs.rename(oldPath, newPath, callback)`

Node.js Database – MySQL Module

You have to install MySQL driver to access a MySQL database with Node.js.

To download and install the "mysql" module, open the Command Terminal and execute the following:

```
npm install mysql
```

Example

```
var mysql = require('mysql');
var con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "root",
  database:"webapp"
});

//Create Connection
/*
con.connect(function (err) {
  if (err) throw err;
  console.log("Connected!");
});

*/

con.connect(function (err) {
  if (err) throw err;
  con.query("SELECT * FROM user", function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
con.end;
```

```
C:\NodeApp>node index.js
[
  RowDataPacket {
    id: 1,
    username: 'satya',
    password: 'sasas',
    email: '123',
    status: 'active'
  },
  RowDataPacket {
    id: 2,
    username: 'satya',
    password: 'smlcodes',
    email: '111',
    status: 'active'
  },
  RowDataPacket {
    id: 3,
    username: 'satya',
    password: 'satya',
    email: 'smlcodes@gmail.com',
    status: 'active'
  }
]
```

Similarly, for MongoDB, we need to use 'mongodb' module.

```
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/MongoDatabase";
MongoClient.connect(url, function (err, db) {
  if (err) throw err;
  console.log("Database created!");
  db.close();
});

var http = require('http');
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/MongoDatabase";
MongoClient.connect(url, function (err, db) {
  if (err) throw err;
  db.collection("employees").findOne({}, function (err, result) {
    if (err) throw err;
    console.log(result.name);
    db.close();
  });
});
```

Events Module

https://www.tutorialsteacher.com/nodejs/nodejs-eventemitter

Node.js – Frameworks

There are various third party open-source frameworks available in Node Package Manager which makes Node.js application development faster and easy.

The following table lists frameworks for Node.js., Many more there… above are few of them.

Open-Source Framework	Description
Express.js	Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. This is the most popular framework as of now for Node.js.
Mojito	This HTML5 framework for the browser and server from Yahoo offers direct MVC access to the server database through the local routines. One clever feature allows the code to migrate. If the client can't run JavaScript for some reason, Mojito will run it on the server -- a convenient way to handle very thin clients.
Restify	Restify is a node.js module built specifically to enable you to build correct REST web services.
Chocolate.js	Chocolate is a simple webapp framework built on Node.js using Coffeescript.

Node.js	AngularJS
You have to install Node.js on your computer system to use it further for creating web or chat applications.	You have to add the AngularJS file just like any other JavaScript file to use it in applications. It doesn't need to be installed separately.
Node.js supports non-blocking input output I/O and follows an event driven architecture. It is used to create real-time applications such as instant messaging or chat apps.	AngularJS is completely written in JavaScript. It is mainly used to create single-page client-side applications.
Node.js is a platform built on the top of Google's V8 JavaScript engine.	AngularJS is an open source framework, follows the syntax of JavaScript and developed by Google.
Node.js is written in C, C++ and JavaScript languages.	AngularJS is written completely in JavaScript, but it is different from other web application frameworks like jQuery.
Node.js has a lot number of frameworks such as Express.js, Sails.js, Partial.js etc.	AngularJS itself is a web application framework of JavaScript.

Difference between Node.js and AngularJS

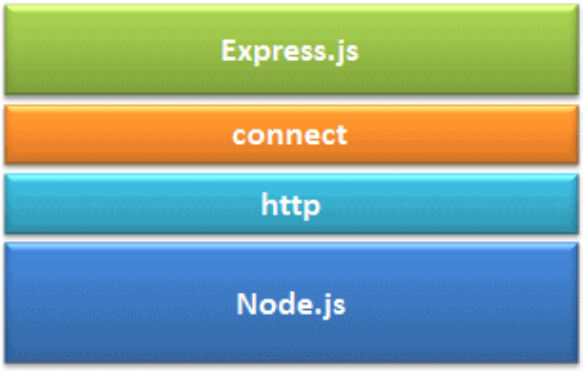
Ref.

https://www.tutorialsteacher.com/nodejs/nodejs-tutorials

2. Express.js

Express.js is a web application framework for Node.js. It provides various features that make web application development fast and easy which otherwise takes more time using only Node.js.

Express.js is based on the **Node.js** middleware module called `connect` which in turn uses `http` module. So, any middleware which is based on `connect` will also work with Express.js.



Advantages of Express.js

1. Makes Node.js **web application development fast and easy**.
2. Allows you to define routes of your application based on HTTP methods and URLs.
3. Includes various middleware modules which you can use to perform additional tasks on request and response.
4. Allows you to define an **error handling middleware**.
5. **Easy to serve static files** and resources of your application.
6. Allows you to create **REST API server**.
7. Easy to connect with databases such **as MongoDB, Redis, MySQL**

Installation

You can install express.js using **npm**. The following command will install latest version of express.js globally on your machine so that every Node.js application on your machine can use it.

```
npm install -g express
```

```
npm install express-generator -g (Optional – Recommended.)
```

Express.js vs Node.js

Node.js	Express.js
It is used to build **server-side, input-output, event-driven** apps.	It is used to build **Web Applications** using approaches and principles of Node.js.
It is built on Google's V8 engine.	It is built on **the top of Node.js.**
Written in C, C++, JavaScript	Written in JavaScript
It is a Run-time platform or environment designed for server-side execution of JavaScript.	**It is a Framework,** based on Node.js.
Controllers are not provided. We need to write manually	Controllers are provided. We just directly use them.
Routing is not provided. need to write manually	Routing is provided.
It requires more coding time.	It requires less coding time.

Express.js Web Application

Express.js provides an easy way to create web server and render HTML pages for different HTTP requests by configuring routes for your application.

Express.js - Create Web Server

In Node.js we manually created server by writing code.

```
var http = require('http');

var server = http.createServer(function(req, res) {
  //Logic to handle requests
});

server.listen(5000);
```

But in Express.js we don't need to write that much code, to create server & request processing. We need to just import **express** module & use the predefined methods.

Example to Create new server

```
var express = require('express');
var app = express();

// define routes here..

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

1.we imported Express.js module using **`require()`** function.

```
var express = require('express');
```

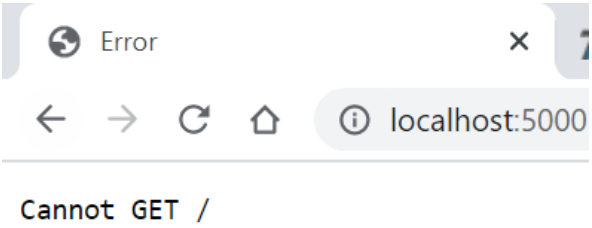
2. Then our application contains express.js related features. Now we need to create an express object, using **`express();`** constructor.

```
var app = express();
```

here the **app** object contains **methods** for routing HTTP requests, configuring middleware, rendering HTML views and registering a template engine.

3. The **`app.listen()`** function creates the Node.js web server at the specified **host and port**. It is identical to Node's **`http.createServer.listen()`** method.

4. Run the above example using **`node app.js`** command and point your browser to **http://localhost:5000**. It will display `Cannot GET /` because we have not configured any routes yet.



Express.js – Routing

Use `app` object to define different routes of your application.

The **app** object includes *`get(), post(), put() and delete()`* methods to define routes for HTTP GET, POST, PUT and DELETE requests respectively.

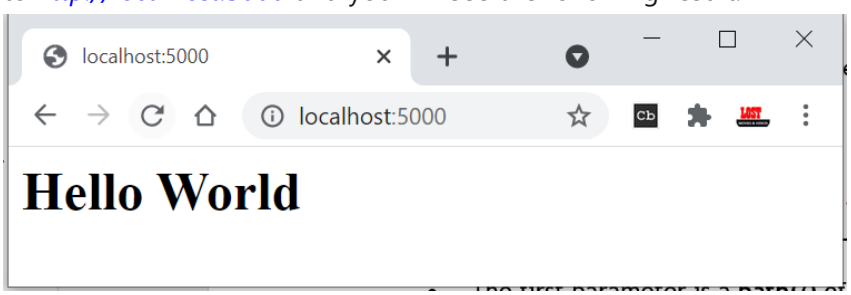
```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('<html><body><h1>Hello World</h1></body></html>');
});
```


Satya Kaveti

```
app.post('/post', function (req, res) {
  res.send('POST Request');
});

app.put('/put', function (req, res) {
  res.send('PUT Request');
});

app.delete('/delete', function (req, res) {
  res.send('DELETE Request');
});

var server = app.listen(5000, function () {
  console.log('Node server is running..');
});
```

- The first parameter is a **path(/)** of a route which will start after base URL.
- The callback function includes `request` and `response` object which will be executed on each request.

Run the above example using **`node server.js`** command and point your browser to *http://localhost:5000* and you will see the following result.



But for *`get(), post(), put() and delete()`* You still get **`Cannot GET /put`** because you are **not handling those requests properly.**

You need to send data to those request types. Next section will see how to send data using Forms.

Handle POST Request

you will learn how to handle HTTP POST request and get data from the submitted form.

To handle HTTP POST request in Express.js version 4 and above, you need to install middleware module called **`body-parser`**. This body-parser module parses the `JSON, buffer, string and url encoded data` submitted using HTTP POST request.

Install body-parser using NPM as shown below.

```
npm install body-parser --save
```


Satya Kaveti

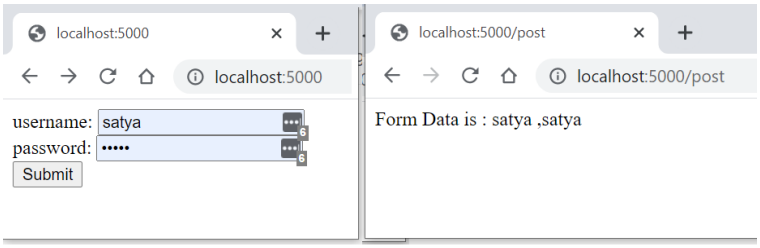
First, create `index.html` file in the root folder of your application and write the following code in it.

```
<form action="/post" method="post">
    username: <input name="username" type="text" /> <br />
    password: <input name="password" type="password" /> <br />
    <input type="submit" />
</form>
```

Create `app.js` file, import **`body-parser`** and get the POST request data as shown below.

```
var express = require('express');
var app = express();

//Import body parser
var bodyParser = require("body-parser");
app.use(bodyParser.urlencoded({ extended: false }));


//set Homepage - index.html
app.get('/', function (req, res) {
    res.sendFile(__dirname + '/index.html');
});


//process form data
app.post('/post', function (req, res) {
    var name = req.body.username + ' ,' + req.body.password;
    res.send('Form Data is : '+name);
});

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

In the above example, POST data can be accessed using **`req.body`**. The **`req.body`** is an object that includes properties for each submitted form.



Express.js - Serving Static Content

In this section, you will learn how to serve static resources like `images, css, JavaScript or other static files.`

Using **`express.static()`** method, you can server static resources directly by specifying the folder name where you have stored your static resources.

The following example serves static resources from the **public** folder under the root folder of your application

```
└──────public
        index.html
        signin.html
        logo.png
```

```
var express = require('express');
var app = express();

//setting middleware
app.use(express.static(__dirname + 'public')); //Serves resources from public folder


var server = app.listen(5000);
```

Express.js – Database Access

Node.js (Express.js on top of Node.js) supports all kinds of databases no matter if it is a relational database or NoSQL database.

To access the database from Node.js, you first need to install drivers for the database you want to use.

The following table lists important **relational databases** and respective drivers.

Relational Databases	Driver	NPM Command
MS SQL Server	**mssql**	`npm install mssql`
Oracle	**oracledb**	`npm install oracledb`
MySQL	**MySQL**	`npm install mysql`
PostgreSQL	**pg**	`npm install pg`
SQLite	**node-sqlite3**	`npm install node-sqlite`

The following table lists important **NoSQL databases** and respective drives.

NoSQL Databases	Driver	NPM Command
MongoDB	mongodb	`npm install mongodb`
Cassandra	cassandra-driver	`npm install cassandra-driver`
LevelDB	leveldb	`npm install level levelup leveldown`
RavenDB	ravendb	`npm install ravendb`
Neo4j	neo4j	`npm install neo4j`
Redis	redis	`npm install redis`
CouchDB	nano	`npm install nano`

Express.js – MySQL Connection

Install MySQL driver

```
npm install mysql --save
```

Example

```
var express = require('express');
var mysql = require('mysql');
var app = express();

//Create MySQL DB Connection
var connection = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'root',
    database: 'webapp'
});


app.get('/', function (req, res) {
    connection.connect(function (error) {
        if (!!error) {
            console.log(error);
        } else {
            connection.query('SELECT * FROM user ORDER BY id desc', function (err, rows) {
                if (err) {
                    console.log(err);
                } else {
                    res.send(rows);
                }
            });
        }
    });
});

var server = app.listen(5000, function () {
    console.log('Server is running..');
});
```

Express.js – MongoDB Connection

In order to access MongoDB database, we need to install MongoDB drivers

```
npm install mongodb --save
```

Connecting MongoDB

```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

     if(err) throw err;

     //Write databse Insert/Update/Query code here..
});
```

Insert Documents

```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

    db.collection('Persons', function (err, collection) {
        collection.insert({ id: 1, firstName: 'Steve', lastName: 'Jobs' });
        collection.insert({ id: 2, firstName: 'Bill', lastName: 'Gates' });
        collection.insert({ id: 3, firstName: 'James', lastName: 'Bond' });


        db.collection('Persons').count(function (err, count) {
            if (err) throw err;
            console.log('Total Rows: ' + count);
        });
    });
});
```

Select Query

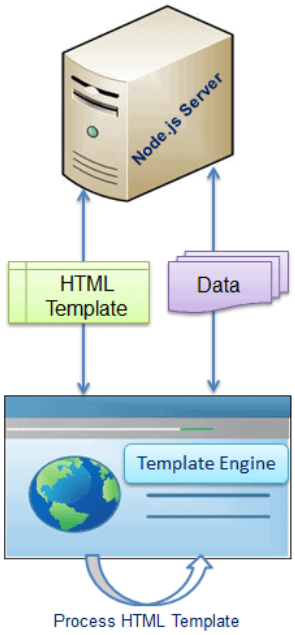
```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

    db.collection('Persons', function (err, collection) {

         collection.find().toArray(function(err, items) {
            if(err) throw err;
            console.log(items);
        });
    });
});
```

Similarly, we do Update/Delete as well.

Node.js - Template Engines

Template engine helps us to create an HTML template with minimal code. Also, it can inject data into HTML template at client side and produce the final HTML.

The following figure illustrates how template engine works in Node.js. Template engine produces the final HTML using **template** and **data** in client's browser.



The following is a list of important (but not limited) template engines for Node.js

- Jade
- Vash
- EJS
- Mustache
- Dust.js
- Nunjucks
- Handlebars
- atpl
- haml

3. React JS

Front-end development refers to what the end user (also commonly referred to as the "**client**") can see. In the most basic forms, front-end development consists of HTML, CSS, and JavaScript.

Eventually, developers decided that there must be a better way to manage all of that code, so they created libraries that could make life easier. **React** was one of those libraries.

React was created by Facebook and released to the public in May of 2013 and has been consistently maintained since then.

- React is used to build user interfaces (UI) on the front end.
- React is the **view** layer of an MVC application (Model View Controller)

React - Introduction

React is one of the most popular JavaScript libraries for **front-end web applications**.

Interactive websites need to update the DOM (Document Object Model) each time a change happens. This process is generally resourceful and slow.

Compared to other libraries that manipulate the DOM, React uses a **Virtual DOM**, allowing to update only the parts of the website that have changed. This increases the speed of updates dramatically, as modern web applications may contain thousands of elements.

Installation

To work with React, we don't need to install anything. We just need to add following `js` files.

- **React** - the React top level API
- **React DOM** - adds DOM-specific methods
- **Babel** - a JavaScript compiler that lets us use ES6+ in old browsers

First, we need to add the React library as two **script** tags to the **head** of our HTML document:

```
<script src="https://unpkg.com/react@16/umd/react.development.js" crossorigin></script>
<script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js" crossorigin />
```

To enable the use of **JSX, use babel**. **JSX** is a syntax extension to JavaScript.

```
<script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
```

Hello, World – Using index.tml

After adding the required script tags, we can start building our React app!

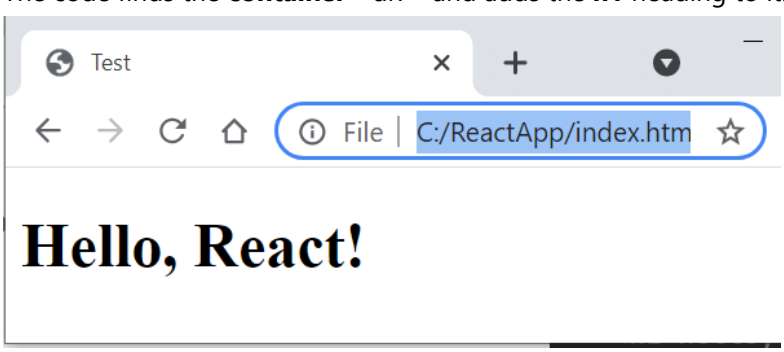
We need to add a container to display something using React.

```
<div id="container"></div>
```

ReactDOM.render(data, element)

```
<!DOCTYPE html>
<html>
    <head>
        <title>Test</title>
        <script src="https://unpkg.com/react@16/umd/react.development.js" crossorigin/>
        <script src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js" crossorigin></script>
        <script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
    </head>
    <body>
        <div id="container"></div>
        <script type="text/babel">

        ReactDOM.render(
          <h1>Hello, React!</h1>,
          document.getElementById('container')
        )

        </script>
    </body>
</html>
```

The code finds the **container** and adds the **h1** heading to it.



HelloWorld – Using Class

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/react@16/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  </head>
  <body>
    <div id="root"></div>
```

```
    <script type="text/babel">
      class App extends React.Component {
        render() {
          return <h1>Hello Satya!</h1>
        }
      }

      ReactDOM.render(<App />, document.getElementById('root'))
    </script>
  </body>
</html>
```

In above we are **`return`** **single line** content, if we have **multiple lines** – we need to use one of below.

1.using ( ) brackets

```
    render() {
        return (
        <div className="App">
            <h1>Hello, React!</h1>
         </div>
        );
      }
```

2.using [ ] – as Array of elements

```
      render() {
        return [
          <ChildA key="key1" />,
          <ChildB key="key2" />,
          <ChildC key="key3" />,
        ];
      }
```

React - Create React App

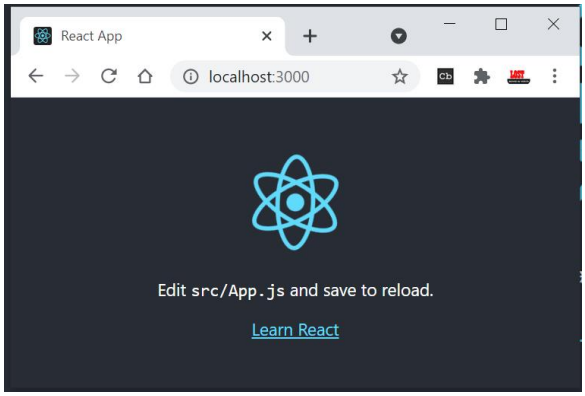
In the previous lesson we learned how to add React to a simple HTML document using the script tags. However, real web apps have a different scale, contain multiple files, use 3rd party libraries, etc.

Facebook has created a handy tool called **`create-react-app`** that makes it easy to setup a React project with just a simple command. **Create React App** allows us to focus on the code, rather than installing and configuring different tools.

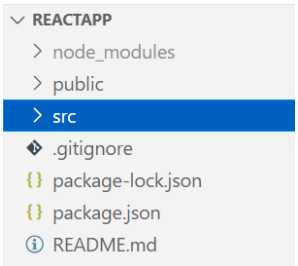
To get started, make sure you have a recent version of **Node** installed on your machine.
Run the following commands in the Terminal to create and start a React app called "**ReactApp**":

```
npx create-react-app reactapp
cd reactapp
npm start
```

This will install all the required dependencies, configure and start the project on **localhost:3000**.
This is the default output of our project in the browser:



Folder Structure



`/public` folder contains files related to how the application will display on the client, the most important of those being **index.html**, which is the HTML template of our page.

`/src` folder contains all of the `JavaScript, CSS, and image files` that will be compiled into a bundle file and injected into **index.html.**

The two files below are the only critical files:

- `index.js:` This file is the entry point into our application. In our code, a method called `ReactDOM.render()` is used to find an element with `id="root"` in the HTML and add content to it.

- `app.js`: This file is the main **component** that will be rendered to the DOM, which currently includes the React logo image and the default text, that we see in the output.

Webpack creates a "bundle" file containing the content of multiple files that need to be "bundled" together and it is all added together into a single file.

Customizing React App

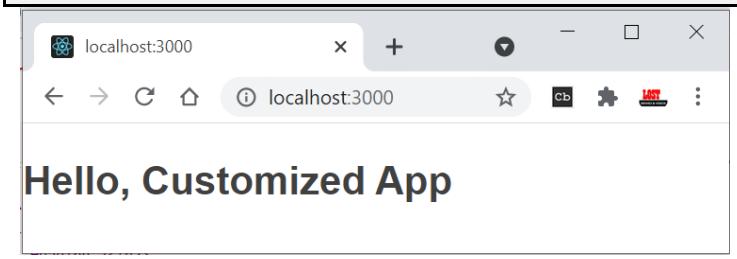
Go ahead and delete all the files out of the `/src` directory, and we'll create our own boilerplate file without any bloat. We'll just keep `index.css` and `index.js.`

Now in **index.js**, we're importing `React, ReactDOM,` and the `CSS file`.

```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'

class App extends React.Component {
    render() {
        return (
        <div className="App">
            <h1>Hello, Customized App</h1>
         </div>
        );
      }
}
ReactDOM.render(<App />, document.getElementById('root'));
```

In **index.html**, just place only & **start** the Application

```
<div id="root"></div>
```



If we use `import React, {Component} from 'react',` No need to write complete extends class name

```
import React, {Component} from 'react'

class App extends Component {

}
```

React - JSX - JavaScript + XML

```
ReactDOM.render(
  <h1>Hello, React!</h1>,
  document.getElementById('root')
);
```

We will start with the **`<h1>Hello, React!</h1>`** element.

As you can see, the element is not in quotes to represent a string. It's like an HTML element, however we use it right in the JavaScript code!

This is called **JSX**, and it is a syntax extension to JavaScript. It allows us to build UI elements right in the JavaScript code!

```
ReactDOM.render(<App />, document.getElementById('root'));
```

The code calls React's **render** method, and passes it two arguments, a **JSX element** and **a container**.

The **`render`** method displays the provided element in the container, which, in our case, is the HTML element with **`id="root".`**

Expressions in JSX

We can use any JavaScript expression inside JSX using curly braces.

For example:

```
const name = "David";
const el = <p>Hello, {name}</p>;

ReactDOM.render(
el,
document.getElementById('root')
);
```

Attributes in JSX

We can specify attributes using quotes, just like in HTML:

```
<div id="name"></div>
```

When using a JavaScript expression as the attributes value, the quotes should not be used:

```
<div id={user.id}></div>
```

Finally, When the JSX expressions are compiled, they are converted into JavaScript **objects**, representing React elements.

React then uses these elements to build the corresponding HTML DOM and display it in the browser.

Virtual DOM

We learned in the previous part that React updates only the elements that are necessary.
This allows React apps to be much faster than apps built with other front-end technologies.

But how does React achieve that?

React uses a **Virtual DOM**, which is a lightweight representation of the DOM.
When an element gets changed, it is first updated in the Virtual DOM. That process is fast, as the virtual DOM is represented by simple objects.

After that, React compares the Virtual DOM to its previous state and only applies the DOM updates necessary to bring the DOM to the desired state.

React – Components

So far, we've created one component - the **App** component. Most React apps have many small components, and everything loads into the main **App** component.

In React, there are two types of components that you can use:

- **Functional Components**
- **Class Components**.

Functional Components

A functional component is a simple JavaScript function:

```
function Hello() {
  return <h1>Hello world.</h1>;
}
```

The code above defined a functional component called **Hello**, that returns a simple React element.

Notice that the name of the functional component begins with a **capital letter**. **This is absolutely critical**. If we start the name of a component with a lowercase letter, the browser will treat our component like a regular HTML element instead of a Component.

In order to display the component, we need to create the corresponding JSX element, by calling the function.

For example, for our user-defined component **Hello**:

```
const el = <Hello/>;
```

Now, we can use our user-defined element and render it on the page:

```
function Hello() {
  return <h1>Hello world.</h1>;
}

const el = <Hello />;
ReactDOM.render(
  el,
  document.getElementById('root')
);
```

Remember, all component names need to start with a capital letter.

Class Components

Class components are typically used when there are more advanced user interactions, like forms, and animations.

All class components need to extend the **React.Component** class. Class components need to have a **render()** method, which is in charge of telling what the page should show.

Remove the **App** class from **index.js**, place it in new file called **app.js**

```
//app.js
import React, {Component} from 'react'

class App extends Component {
    render() {
        return (
        <div className="App">
            <h1>Hello, Customized App</h1>
         </div>
        );
      }
}
export default App
```

Import in **index.js**

```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'
import App from './app'

 //Code Removed

ReactDOM.render(<App />, document.getElementById('root'));
```

We export the component as **App** and load it in **index.js.**

In **index.html**, just place only & **start** the Application

```
<div id="root"></div>
```

Example Simple Component

simple component, which is a function. This component doesn't use the **class** keyword. Let's take our **Table** and make two simple components for it - a **TableHeader**, and a **TableBody**.

We're going to use ES6 arrow functions to create these simple components.

First, the **TableHeader**

```
const TableHeader = () => {
    return (
      <thead>
        <tr>
          <th>Name</th>
          <th>Job</th>
        </tr>
      </thead>
    )
  }
```

Then the **TableBody** .

```
  const TableBody = () => {
    return (
      <tbody>
        <tr>
          <td>Charlie</td>
          <td>Janitor</td>
        </tr>
        <tr>
          <td>Mac</td>
          <td>Bouncer</td>
        </tr>
      </tbody>
    )
  }
```

Now our Table file will look like this. Note that the **TableHeader** and **TableBody** components are all in the same file and being used by the Table class component.

```
const TableHeader = () => { ... }
const TableBody = () => { ... }
class Table extends Component {
  render() {
    return (
      <table>
        <TableHeader />
        <TableBody />
      </table>
    )
  }
}
```

Example Class Component

We are created Table component to display table data in **table.js** – and export as **Table** component

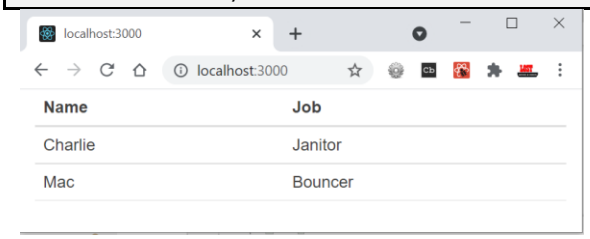
```
import React, {Component} from 'react'

class Table extends Component {
  render() {
    return (
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Job</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>Charlie</td>
            <td>Janitor</td>
          </tr>
        </tbody>
      </table>
    )
  }
}
export default Table
```

Import Table in **app.js** – pass it to index.html

```
import React, {Component} from 'react'
import Table from './table'

class App extends Component {
  render() {
    return (
      <div className="container">
        <Table />
      </div>
    )
  }
}
export default App
```

In **index.html**, just place only & **start** the Application

```
<div id="root"></div>
```



React – Data Handling

To handle data in React, we do with properties, referred to as **props**, and with **state**

Props

Functional components can accept arguments, similar to JavaScript functions. These arguments are called **props**, and represent an object.

For example, we can use **props** in our Hello component:

```
function Hello(props) {
  return <p>Hello, {props.name}!</p>;
}
```

Props can be accessed in class components using **this.props**.

```
class Hello extends React.Component {
  render() {
    return <p>Hello, {this.props.name}!</p>;
  }
}
```

An important thing to consider is that props are **read-only**, meaning components cannot modify their props.

State

Many web apps need their components to change their data, for example, after user interaction (clicking a button, submitting a form, etc.).However, **props** cannot be changed.

In order to allow components to manage and change their data, React provides a feature called **state**. **State** is an object that is added as a property in class components.

State is just a simple object, that contains `key:value` pairs.Similar to props, the values can be accessed using **this.state**.

```
class Hello extends React.Component {
  state = {
    name: "James"
  }
  render() {
    return <h1>Hello {this.state.name}.</h1>;
  }
}
```

Changing State

```
this.setState({
name: "James",
age: 25
});
```

React - Lifecycle Methods

`constructor (props)`	Before rendering #
`componentWillMount()`	Don't use this #
`render()`	Render #
`componentDidMount()`	After rendering (DOM available) #
`componentWillUnmount()`	Before DOM removal #
`componentDidCatch()`	Catch errors (16+) #

Set initial the state on `constructor()`. Add DOM event handlers, timers (etc) on `componentDidMount()`, then remove them on `componentWillUnmount()`.

React – Routing

To use React Router, you first have to install it using NPM:

```
npm install react-router-dom
```

You'll need to import **BrowserRouter, Route, and Switch** from **react-router-dom** package:

```
import React, { Component } from 'react';
import { BrowserRouter, Route, Switch } from 'react-router-dom';
```

For every request it goes through **<BrowserRouter>. <App/>** component does all the logic of displaying various components that you provide it with.

```
// index.js
ReactDOM.render(
    <BrowserRouter>
        <App />
    </BrowserRouter>,
    document.getElementById('root')
)
```

Now in App component (**App.js**) - Configure Routes. We use **<Switch>** , inside that **<Route>** to configure different routes

```
function App() {
    return (
        <main>
            <Switch>
                <Route path="/" component={Home} exact />
                <Route path="/about" component={About} />
                <Route path="/shop" component={Shop} />
                 <Route component={Error} />
            </Switch>
        </main>
    )
}
```

- **<Switch>** element ensure that only one component is rendered at a time.
- **<Route>** tag contains the Path & the component need to load on particular Path
- **{Error}** just like a normal `<Route>` tag, but with no path. Error component contains `<h1>Oops! Page not found!</h1>` if a user types an incorrect URL, it will display error message

This is Now **import** the components into the app. You may wish to have them in a separate "components" folder to keep code clean and readable.

```
import Home from './components/Home';
import About from './components/About';
import Shop from './components/Shop';
```

So far, our site is only navigable by typing the URLs. To add clickable links to the site, we use the `Link` element from React Router and set up a new `Navbar` component.

```
function Navbar() {
  return (
    <div>
      <Link to="/">Home </Link>
      <Link to="/about">About Us </Link>
      <Link to="/shop">Shop Now </Link>
    </div>
  );
};
```

Ref.

https://www.sololearn.com/learning/1097/3370/7610/2

_Best
https://www.taniarascia.com/getting-started-with-react/

Ref.
https://www.freecodecamp.org/news/react-router-in-5-minutes/

Enough, from now onwards read
https://www.w3schools.com/react/react_props.asp

4. Type Script

Introduction to TypeScript

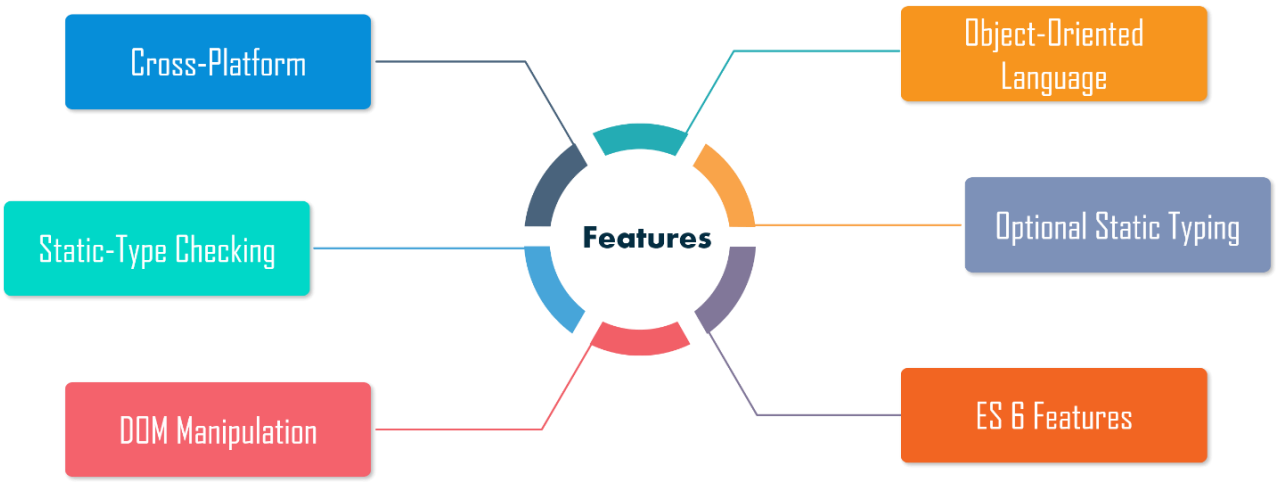
TypeScript is a typed superset of JavaScript that compiles to plain JavaScript. TypeScript is pure object-oriented with classes, interfaces and statically typed programming languages like C# or Java. It requires a compiler to compile and generate in JavaScript file. Basically, TypeScript is the ES6 version of JavaScript with some additional features.

A TypeScript code is written in a file with **.ts extension** and then compiled into JavaScript using the compiler.

After the installation, the command **tsc <filename>.ts** compiles the TypeScript code into a plain JavaScript file.

```
Syntax:
var message:string = "Welcome to Edureka!"
console.log(message)

// On compiling, it generates the following JavaScript code:
var message = "Welcome to Edureka!";
console.log(message);
```

Features of TypeScript



- **Cross-Platform:** The TypeScript compiler can be installed on any Operating System such as Windows, MacOS and Linux.
- **Object-Oriented Language**: TypeScript provides features like Classes, Interfaces, and Modules. Thus, it can write object-oriented code for client-side as well as server-side development.
- **Static type-checking**: TypeScript uses static typing and helps type checking at compile time. Thus, you can find errors while writing the code without running the script.

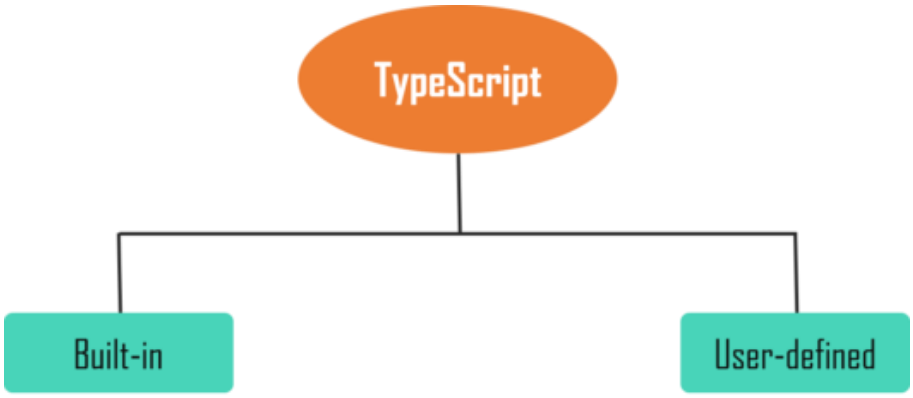
- **Optional Static Typing**: TypeScript also allows optional static typing in case you are using the dynamic typing of JavaScript.
- **DOM Manipulation**: You can use TypeScript to manipulate the DOM for adding or removing elements.
- **ES 6 Features**: TypeScript includes most features of planned ECMAScript 2015 (ES 6, 7) such as class, interface, Arrow functions, etc.

Advantages of Using TypeScript

- TypeScript is **fast, simple, easy to learn** and runs on any browser or JavaScript engine.
- It is **similar** to **JavaScript** and uses the same syntax and semantics.
- This helps backend developers write front-end **code faster**.
- TypeScript code can be called from an **existing JavaScript code**. Also, it works with existing JavaScript frameworks and libraries without any issues.
- The Definition file, with .ts extension, provides support for existing JavaScript libraries like **Jquery, D3.js**, etc. So, TypeScript code can add JavaScript libraries using type definitions to avail the benefits of type-checking, code autocompletion, and documentation in existing dynamically-typed JavaScript libraries.
- It includes features from **ES6** and **ES7** that can run in ES5-level JavaScript engines like Node.js.

Data Types

The Type System represents the different types of values supported by the language. It checks the **validity** of the supplied **values** before they are stored or manipulated by the program.



It can be classified into two types such as:

- **Built-in**: This includes **number, string, boolean, void, null and undefined**.
- **User-defined**: It includes **enums, classes, interfaces, arrays, and tuple.**

Variables

A variable is a named space in the memory which is used to store values. The type syntax for declaring a variable in TypeScript includes a colon **(:)** after the variable name, followed by its type. Similar to JavaScript, we use the var keyword to declare a variable.

There are four options when we declare a variable:

```
var [identifier] : [type-annotation] = value;
var [identifier] : [type-annotation];
var [identifier] = value;
var [identifier];
```

Example:

```
var name:string = "Daisy";
var empid:number = 1001;
console.log("name"+name)
console.log("employee id "+empid)
```

On compiling, it will generate following JavaScript code:

```
//Generated by typescript 1.8.10
var name = "Daisy";
var empid = 1001;
console.log("name" + name);
console.log("employee id: " + empid);
```

Output:

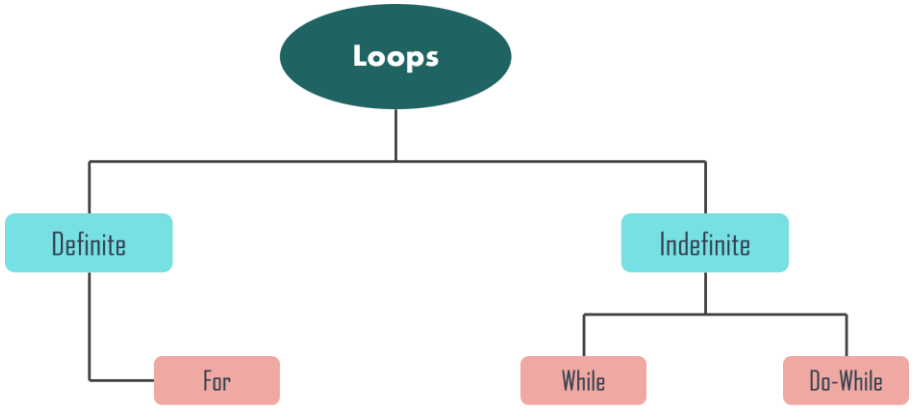
```
name:Daisy
employee id:1001
```

Now let's move on to the next topic of our TypeScript Tutorial.

Loops

There may be situations when a block of code needs to be executed several numbers of times. A loop statement allows us to execute a statement or group of statements multiple times.

The TypeScript loops can be classified as:



For Loop

The for loop is an implementation of a definite loop.

```
Syntax:
for (first expression; second expression; third expression ) {
// statements to be executed repeatedly
}
```

Here, the first expression is executed before the loop starts. The second expression is the condition for the loop to execute. And the third expression is executed after the execution of every code block.

Example:

```
for (let i = 0; i < 2; i++) {
console.log ("Execute block statement" + i);
}
```

While Loop

The while loop executes the instructions each time the condition specified evaluates to true.

Syntax:

```
while (condition expression) {
// code block to be executed
}
```

Example:

```
let i: number = 1;

while (i < 3) {
console.log( "Block statement execution no." + i )
i++;
}
```

Functions

TypeScript provides the concept of classes and modules, functions still are an integral part of the language.

Named Functions

A named function is used to declare and call a function by its given name.

Example:

```
function display() {
console.log("TypeScript Function");
}

display();
```

Output: TypeScript Function

Anonymous Function

An anonymous function is one which is defined as an expression. This expression is stored in a variable. These functions are invoked using the variable name that the function is stored in.

Example:

```
let greeting = function() {
console.log("TypeScript Function");
};

greeting();
```

Output: TypeScript Function

Arrow Function

Fat arrow notations are used for anonymous functions i.e for function expressions. They are also called lambda functions in other languages.

```
Syntax:
(param1, param2, ..., paramN) => expression
```

Using the fat arrow (=>) drops the need to use the 'function' keyword. Parameters are passed in the angular brackets <>, and the function expression is enclosed within the curly brackets {}.

Example:

```
let sum = (x: number, y: number): number => {
return x + y;
}

sum(10, 30); //returns 40
```

Function Overloading

TypeScript provides the concept of function overloading. Thus, you can have multiple functions with the same name but different parameter types and return type.

Example:

```
function add(a:string, b:string):string;

function add(a:number, b:number): number;
    return a + b;
}

add("Hello ", "Edureka"); // returns "Hello Edureka"
add(10, 10); // returns 20
```

Strings

The string is another primitive data type that is used to store text data. String values are surrounded by single quotation marks or double quotation marks.

Syntax

```
var var_name = new String(string);
```

There are different properties of methods available in String object such as:

- **Constructor** – It returns a reference to the String function that created the object
- **Length** – This returns the length of the string
- **Prototype** – This property allows you to add properties and methods to an object

```
let name = new String("Welcome to Edureka!");
console.log("Message: " +name);
console.log("Length: "+name.length);

Output:
Message: Welcome to Edureka!
Length: 19
```

Arrays

An array is a special type of data type that stores multiple values of different data types sequentially using a special syntax. Array elements are identified by index of the element.

Syntax:

```
var array_name[:datatype]; //declaration
array_name = [val1,val2,valn..] //initialization
```

```
let names: Array<string>;
names = ['John', 'Daisy', 'Rachel'];

let ids: Array<number>;
ids = [101, 700, 321];
```

Interfaces

The interface contains only the declaration of the data members & methods.

Example:

```
interface Employee {
empID: number;
empName: string;
getSalary: (number) => number; // arrow function
getManagerName(number): string;
}
```

In the above example, the **Employee** interface includes two properties **empID** and **empName**. It also includes a method declaration **getSalaray** using an **arrow function** which includes one number parameter and a number return type. The **getManagerName** method is declared using a normal function.

Classes

TypeScript introduced classes so that they can avail the benefits of object-oriented techniques like encapsulation and abstraction. The class in TypeScript is compiled to plain JavaScript functions by the TypeScript compiler to work across platforms and browsers.

A class includes the following:

- **Constructor**
- **Properties**
- **Methods**

Example:

```
class Employee {
empID: number;
empName: string;

constructor(ID: number, name: string) {
this.empName = name;
this.empID = ID;
}

getSalary() : number {
return 40000;
}
}
```

Inheritance

TypeScript supports Inheritance as it is the ability of a program to create new classes from an existing class. The class that is extended to create newer classes is called the parent class or the super class. The newly created classes are called the child or sub classes.

A class inherits from another class using the 'extends' keyword. Child classes inherit all properties and methods except private members and constructors from the parent class. But, TypeScript does not support multiple inheritance.

Syntax:

```
class child_class_name extends parent_class_name
```

Example:

```
class Person {
name: string;
        constructor(name: string) {
                this.name = name;
        }
}
-----------------------------------------------------------------------------------------
class Employee extends Person {
empID: number;

constructor(empID: number, name:string) {
        super(name);
        this.empID = empid;
}

displayName():void {
        console.log("Name = " + this.name + ", Employee ID = " + this.empID);
}
}

let emp = new Employee(701, "Jason");
emp.displayName(); // Name =Jason, Employee ID = 701
```

Objects

An object is an instance that contains a different set of key-value pairs. The values can be scalar values or functions or even an array of other objects.

Syntax:

```
var object_name = {
key1: “value1”, //scalar value
key2: “value”,
key3: function() {
//functions
},
key4:[“content1”, “content2”]
```

An object can contain scalar values, functions and structures like arrays and tuples.

Example:

```
var person = {
firstname:"Danny",
lastname:"Green"
};
//access the object values
console.log(person.firstname)
console.log(person.lastname)
```

On compiling, it will generate the same code in JavaScript.

```
Danny
Green
```

Ref. https://www.edureka.co/blog/typescript-tutorial/

5. Angular

Angular is an open-source JavaScript MVC framework for the web applications. It extends the HTML and makes it dynamic**. Angular is used to create Single Page Applications.**

AngularJS or **Angular 1.x** was the first iteration of the Angular framework. With the release of **Angular 2**, Angular was completely redesigned using TypeScript and component-based architecture.

It is important to know that when developers say **Angular**, they mean **Angular** versions **2 and higher**. **AngularJS** refers to version 1 of Angular only.

Install Angular

Install Node.js

Angular uses NPM (Node Package Manager) to install libraries, packages. So, you need to install NPM before installing Angular.

```
C:\Windows\system32>node -v
v14.16.1

C:\Windows\system32>npm -v
7.19.1
```

After installing Node.js and NPM, install the Angular CLI.

Install Angular CLI

Angular CLI (Command Line Interface) used to install libraries required for your application. Angular CLI is also used to generate, build, run, and deploy Angular application

To install the Angular CLI globally using NPM, open a terminal/command window, and enter the following command:

```
npm install -g @angular/cli
```

check the Angular version using **ng v**

```
C:\Windows\system32>ng v

Angular CLI: 12.1.1
Node: 14.16.1
Package Manager: npm 7.19.1
OS: win32 x64
```

TypeScript – Datatypes

AngularJS used Javascript as its primary language. Angular began using TypeScript starting in Angular 2. Since then, TypeScript has become the primary language of Angular.

TypeScript uses JavaScript's `var, let,` and `const` keywords to initialize variables. It also uses a colon `(:)` after the variable name to designate the type.

TypeScript does not require you to statically type variables. Plain Javascript syntax WILL work in TypeScript. However, by convention, most programmers statically type variables in TypeScript because it helps to provide structure and prevent programmer errors.

String

Javascript:

```
let name = 'Fred'
```

TypeScript:

```
let name: string = 'Fred'
```

Based on this convention, we could initialize a `shoeColor` string in TypeScript as such:

```
let shoeColor: string = 'blue';
console.log(shoeColor);
```

It is possible to change **variable** values over time:

```
shoeColor = 'red' ;
console.log(shoeColor);
```

Numbers

We can initialize a `seatCount` number variable in TypeScript like this:

```
let seatCount: number = 42;
```

TypeScript does not designate between integers and floating point or decimal numbers like other languages when it comes to typing. **Initializing a decimal number is just like initializing an integer**:

```
let registrationPercentage: number = 92.87;
```

If you were to try to reassign `registrationPercentage` to a `string` value in another line of that program, the program would not compile, and your text editor would most likely throw an error.

```
registrationPercentage = 'Ninety Two';
```

// WILL CAUSE AN ERROR because registrationPercentage is statically typed as a number.

Boolean

Boolean values are either true or false. In TypeScript, you can declare and statically type a variable without initializing it with a value. Booleans always return **true** or **false** values. These values are not strings, they are actually their own data type.

```
let isRegistered: boolean;
```

If you try to use **isRegistered** at this point in the program, however, you will get an error. We need to initialize with value before using it.

Enums

Enum values in Typescript are collections of constants. Think of an Enum as a type of fixed array of things that you use as a reference, such as days of the week, states in the USA, etc. The number of elements and the order of elements doesn't change either.

Let's create an enum called **spiceLevel:**

```
enum spiceLevel {
NONE = "no spice",
LOW = "barely spicy",
MEDIUM = "medium spicy",
HIGH = "hot"
}

console.log(spiceLevel.MEDIUM);
// outputs "medium spicy"
```

any

The 'any' type is essentially a wild card. **It can hold anything**. We can declare an 'any' data type in TypeScript like this:

```
let userData: any;
```

userData will now be a container that can hold any data type. Once **userData** is initialized, however, it will infer the type from the value given to it.

```
let userData: any;

userData = 22;
console.log(userData + 2);
// logs 24

userData = "free";
console.log(userData + "man");
// logs "freeman";
```

Note that with an **any** data type, even though the type is inferred upon assignment, you can re-assign the variable to another data type and it still works.

Interface

An interface is defined with the keyword `interface` and it can include properties and method declarations using a function or an arrow function.

```
interface IPerson {
   firstName:string,
   lastName:string,
   sayHi: ()=>string
}

var customer:IPerson = {
   firstName:"Tom",
   lastName:"Hanks",
   sayHi: ():string =>{return "Hi there"}
}

console.log("Customer Object ")
console.log(customer.firstName)
console.log(customer.lastName)
console.log(customer.sayHi())

var employee:IPerson = {
   firstName:"Jim",
   lastName:"Blakes",
   sayHi: ():string =>{return "Hello!!!"}
}

console.log("Employee  Object ")
console.log(employee.firstName);
console.log(employee.lastName);
```

We can use **optional** fields in an interface in TypeScript. Optional fields are not part of the strict interface contract. You can omit them when creating an instance of that interface.

```
interface Order {
  customerName: string,
  itemNumbers: number[],
  isComplete?: boolean
}
```

Notice the question mark after isComplete. **isComplete?** means that we can omit that.

Create Angular Application

To create an application, and execute the `ng new <project name>`

```
ng new AngularApp
```

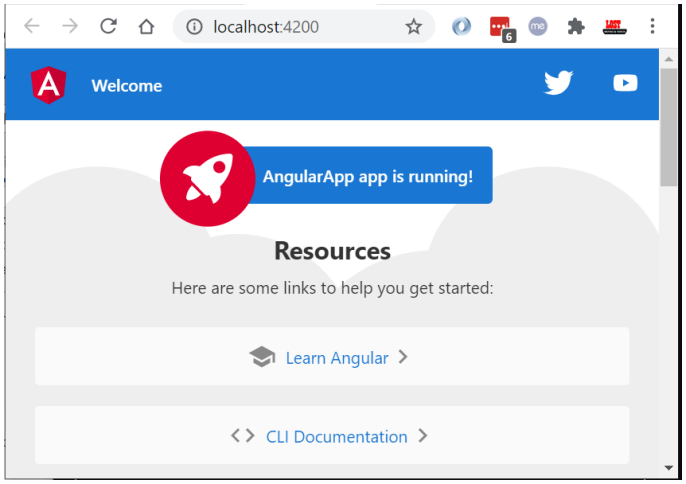
Build Application

```
ng serve -o
```

Start Application

```
npm start
```

Open Application in browser: http://localhost:4200



The `ng serve` command keep watching source files, so if you make any changes in any file of the project, it will rebuild it and refresh the browser automatically to reflect the changes.

Open the project in your favorite editor or IDE and navigate to the `src/app` folder to make some changes to the starter application.

You'll find the implementation of the shell `AppComponent` distributed over three files:

- `app.component.ts`— the component class code, written in TypeScript.
- `app.component.html`— the component template, written in HTML.
- `app.component.css`— the component's private CSS styles.

Open the component class file (app.component.ts) and change the value of the title.

app.component.ts (class title property)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Employee Angular App';
}
```

Open the component template file (app.component.html) and delete the default template generated by the Angular CLI. Replace it with the following line of HTML.

app.component.html (template)

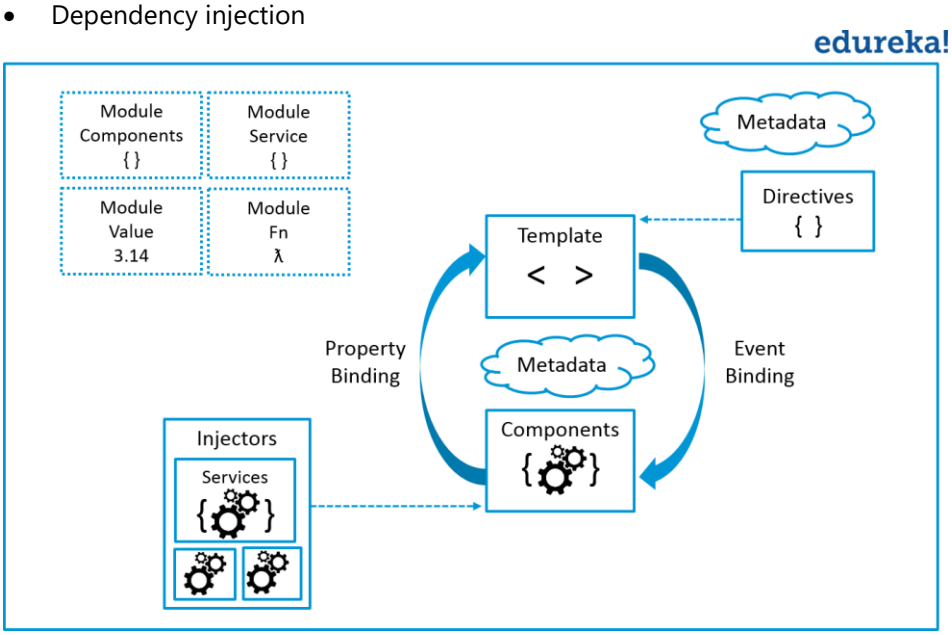
```
<h1>{{title}}</h1>
```


Satya Kaveti

Building Blocks of Angular

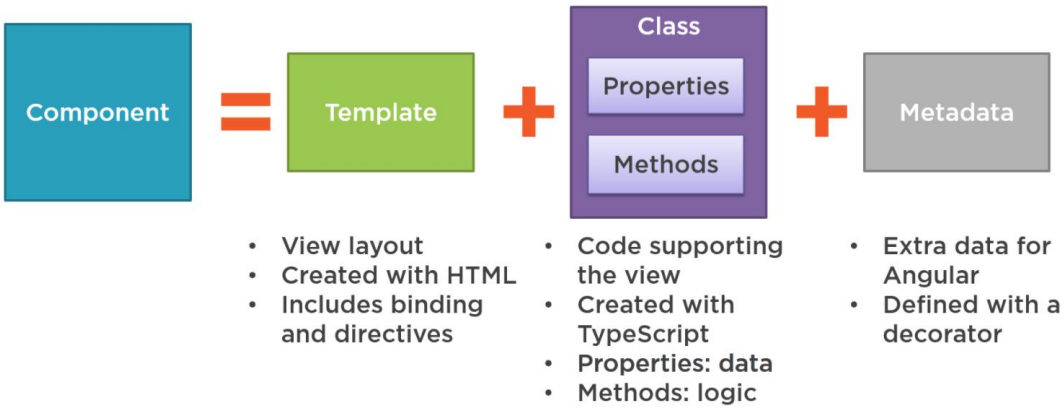
The main building blocks of Angular are:

- Modules
- Components
- Templates
- Metadata
- Data binding
- Directives
- Services
- Dependency injection



Angular Component

- Angular is a SPA framework, and a view is made of one or more component.
- An Angular component represents a portion of a view.
- Angular Component = HTML Template + Component **Class** + Component Metadata



HTML Template

HTML template is nothing but a regular HTML code with additional Angular specific syntax to communicate with the component class.

Class

Essentially, a component class is a TypeScript class that includes **properties** and **methods**.

Properties store data and methods include the logic for the component. Eventually, this class will be compiled into JavaScript.

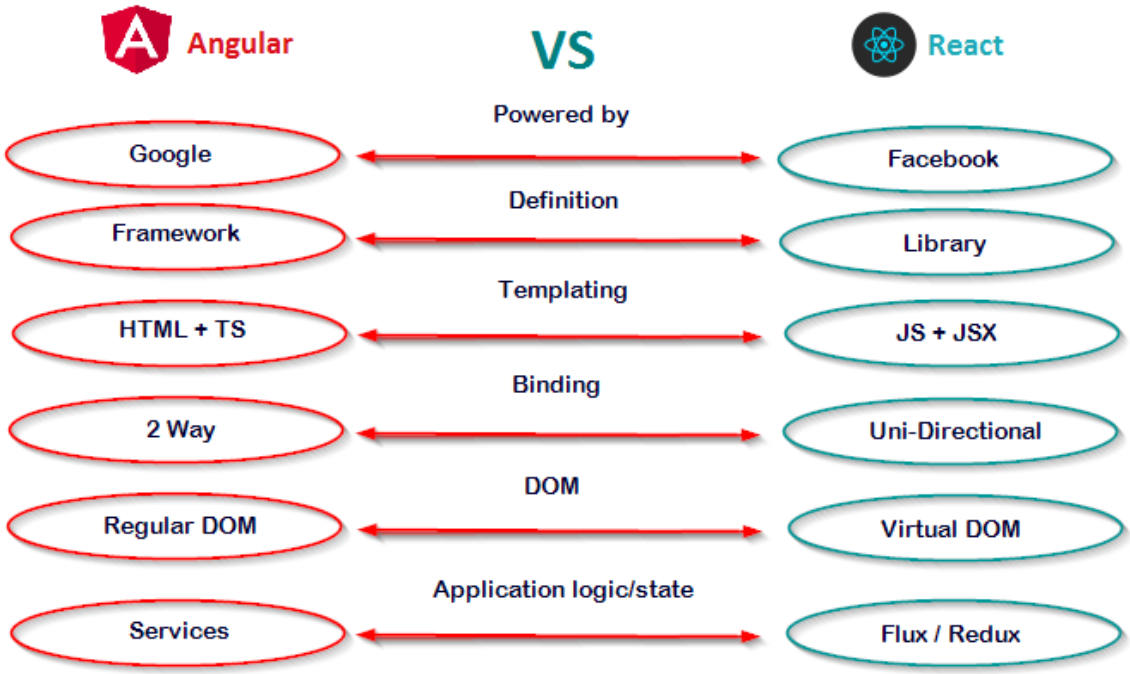
Metadata

Metadata is some extra data for a component used by Angular API to execute the component, such as the location of HTML and CSS files of the component, selector, providers, etc.

Angular vs React.js Comparisons

	Angular	React
Release Year	2016	2013
Type	Framework	Library
Official Site	angular.io	reactjs.org
Developed By	Google	Facebook
License	MIT	MIT
Programming	**TypeScript**	**JavaScript**
Architecture	Component-based	Component-based
Data Binding	Two-way	One-way
DOM	Real DOM	Virtual DOM
Rendering	Client/server side	Client/server side
Template	**HTML+TypeScript**	**JSX+JS**
App size (KB)	~ 500	~ 80
Types of Apps	SPA Hybrid apps Web applications	Native-rendered apps Mobile apps Web applications
Type	Angular is a complete framework.	React is a JavaScript library, and it is much older compared with Angular.
Use of libraries	Angular is a complete solution in itself.	React js can be packaged with other programming libraries.
Learning curve	Learning Angular is not easy for beginners. Thus, it requires lots of training.	It is easier to grasp compared Angular. However, it is difficult to learn when augmented with Redux.

Installation	Angular is easy to set up but may lead to an increase in coding time.	React takes longer to set up. But it is really fast for delivering projects and building apps.
Best feature	It offers a limited amount of freedom and flexibility.	It gives you the freedom to choose the tools, architecture, and libraries, for developing an app.
Data binding	Angular, on the other hand, uses the **two-way** data binding method. It helps you to ensure that the model state automatically changes when any change is made.	React language uses one-way data binding, which means that the UI elements can't be changed without updating the corresponding model state.
Testing	The testing and debugging for a complete project is possible with a single tool.	It requires a set of tools to perform different types of testing.
Documentation	Due to the ongoing development process, the documentation is slower.	Although it is also undergoing regular updates, the documentation is relatively faster.
Application Types	You should use this framework If you want to develop a SPA(Single Page Application) and mobile apps.	Use this app if you want to develop Native apps, hybrid apps, or web apps
Ideal for	Ideal to use when you want to develop large-scale, feature-rich applications.	Ideal for modern web development and native- rendered apps for Android and iOS devices.
Model	Based on MVC (Model View Controller)	It is based on Virtual DOM
Written in	**Typescript**	**JavaScript**
Community	A large community of developers and supporters	Facebook developer's community
Language preference	**TypeScript**	**JSX - JavaScript XML**
Companies Using	Wepay, Beam, Auto Trader, Mesh, Streamline Social, etc.	Facebook, Uber Technologies, Instagram, Netflix, Pinterest, etc.
Template	HTML + TypeScript	JSX + J% (ES5/ES6)
Abstraction	Medium	Strong
Git hub stars	46.6 K	126K
Javascript library	Adding Javascript library to the source code **Not possible**	Adding Javascript library to the source code **Possible**
Restriction	An angular framework is very sensitive, which means that it restricts you from using large models.	React gives you an option to choose without putting any performance penalty.
Use of code	Angular comes with many ready to use elements. However, it mainly comes from a specific provider. So, there are priority collisions and namesDaces.	React allows you to manage the code according to your desired format.
Git hub stars	80.8 K	180K
Fork	48.2 K	30.3 K



Angular	React
Angular architecture is a bit complex; it follows the component-oriented architecture	ReactJS architecture is more simplified; it follows a Model-View-Control (MVC) architecture
Angular is written in Microsoft's TypeScript language	ReactJS is written in JavaScript language
Angular is compatible with any platform	ReactJS is not compatible
The latest version of Angular is Angular 7.2.0	The ReactJS latest version is 16.7.0
Angular supports bidirectional (two-way) data binding	ReactJS supports unidirectional (one-way) data binding
This framework is used in mobile apps and single-page applications	The React framework is used in web applications, hybrid apps, and native applications
In Angular, abstraction is medium	In React, abstraction is strong
Adding a JavaScript library to the source code in Angular is not possible	Adding a JavaScript library to the source code in React is possible
In Angular, documentation is slower	In ReactJS, documentation is faster
A single tool is required for testing a complete project in Angular	More than one tool is required for testing a complete project in React

NODE JS
VERSUS
ANGULAR JS

NODE JS	ANGULAR JS
An open-source, cross-platform JavaScript run-time environment that executes JavaScript code outside of a browser	A JavaScript-based, open-source front-end web application framework that is designed to address changes encountered in developing single page applications
Governed by the Node.js Foundation, is facilitated by the Linux Foundation's Collaborative Projects program	An open source web application development framework developed by Google
Programmer has to install Node JS on his computer to use it	Programmer has to add the Angular JS file similar to a JavaScript file to use it
Written using C, C++, and JavaScript	Written in JavaScript
Has other frameworks such as Sails.js, Express,js etc.	A web framework
Used to build fast and scalable server-side applications	Used to create single page client-side applications

Visit www.PEDIAA.com

https://www.toptal.com/front-end/angular-vs-react-for-web-development

https://mlsdev.com/blog/angular-vs-react

https://programmingwithmosh.com/react/react-vs-angular/

6. Applications for Hands-on

#1 - React + SpringBoot

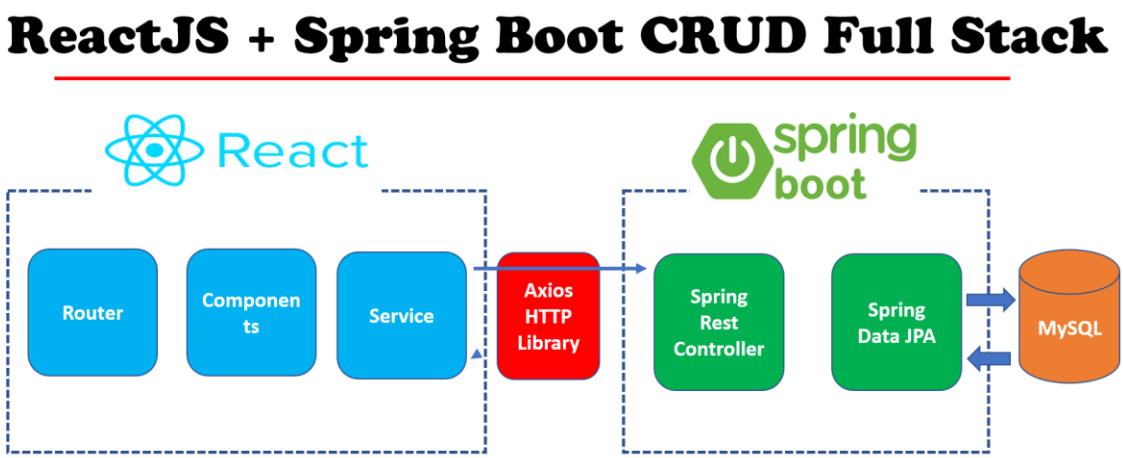
https://github.com/RameshMF/ReactJS-Spring-Boot-CRUD-Full-Stack-App

https://www.javaguides.net/2020/07/spring-boot-react-js-crud-example-tutorial.html

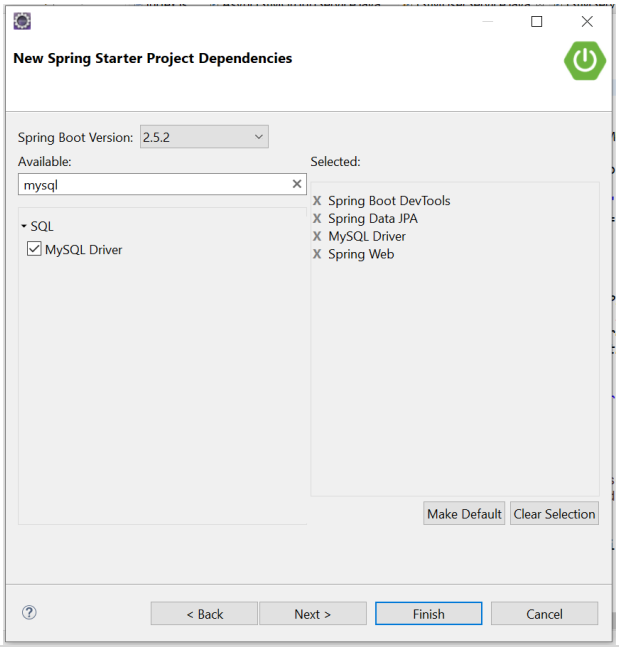
https://www.youtube.com/watch?v=n43h1eJ2EUE&list=PLGRDMO4rOGcNLnW1L2vgsExTBg-VPoZHr

We will build two projects:

1. **sprint boot-backend (server)** – To develop REST API
2. **react-frontend (client)** – Consume REST API



SpringBoot – Employee Backend Service



Satya Kaveti

Configuring MySQL Database

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/webapp?useSSL=false
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5InnoDBDialect
spring.jpa.hibernate.ddl-auto = update
```

```
CREATE TABLE `employee` (
	`id` INT(11) NOT NULL AUTO_INCREMENT,
	`name` VARCHAR(50) NOT NULL DEFAULT '0' COLLATE 'latin1_swedish_ci',
	`address` VARCHAR(50) NOT NULL DEFAULT '0' COLLATE 'latin1_swedish_ci',
	`salary` DOUBLE NULL DEFAULT '0',
	PRIMARY KEY (`id`) USING BTREE
);
```

Model class – Employee.java

```
package springboot.emp.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table
public class Employee {
	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private Long id;

	@Column
	private String name;

	@Column
	private String address;

	@Column
	private Double salary;

	public Employee() {
		super();
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public String getAddress() {
		return address;
	}
	public void setAddress(String address) {
		this.address = address;
```

```
	}
	public double getSalary() {
		return salary;
	}
	public void setSalary(double salary) {
		this.salary = salary;
	}
	public Employee(Long id, String name, String address, Double salary) {
		super();
		this.id = id;
		this.name = name;
		this.address = address;
		this.salary = salary;
	}
	@Override
	public String toString() {
		return "Employee [id=" + id + ", name=" + name + ", address=" + address + ",
salary=" + salary + "]";
	}
}
```

Repository – EmployeeRepository.java

```
package springboot.emp.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import springboot.emp.model.Employee;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long>{

}
```

Exception – EmployeeException

```
package springboot.emp.exception;

public class EmployeeException extends RuntimeException{

	public EmployeeException(String message) {
		super(message);
	}
}
```

Exception – EmployeeController

```
package springboot.emp.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import springboot.emp.model.Employee;
```

```
import springboot.emp.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/v1/")
public class EmployeeController {

        @Autowired
        private EmployeeRepository repository;

        @GetMapping("/all")
        public List<Employee> getAllEmployees(){
                return repository.findAll();
        }
}
```

Test Rest API

http://localhost:8080/api/v1/all

```
[
  {
    "name": "DILEEP",
    "address": "XXXXXXXXX",
    "salary": 0
  },
  {
    "name": "DILEEP",
    "address": "BANGLORE",
    "salary": 0
  }
]
```

React – Employee Front End

1.Create EmployeeReactApp – Empty Repo

Open Visual Studio Code > Terminal

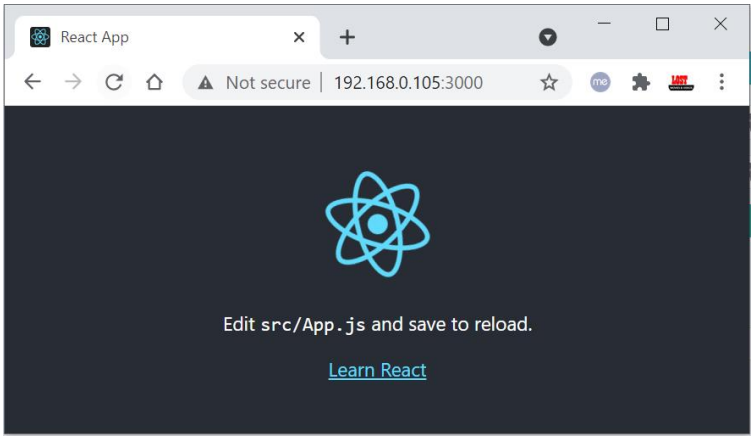
```
npx create-react-app employee-react-app
```

Start react app

```
cd employee-react-app
npm start
```

You can now view employee-react-app in the browser.

```
  Local:            http://localhost:3000
  On Your Network:  http://192.168.0.105:3000
```



Main files

- public/**index.html** – Single page appl'n , contains `<div id="root"></div>`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>

  </body>
</html>
```

- src/**App.js** - Contains Component code to generate UI.

```
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">

      <h1>Hello, World!!!</h1>

      </header>
    </div>
  );
}

export default App;
```

- src/**index.js** - get code from component & replace div content to render UI.

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();
```

3. Get All Employees – React Code

employee-react-app\src\component**ListAllEmployeesComponent.jsx**

```
import React, { Component } from 'react';
import EmployeeService from '../services/EmployeeService';

class ListAllEmployeesComponent extends Component {

    constructor(props) {
        super(props)

        this.state = {
                employees: []
        }

    }

    componentDidMount(){
        EmployeeService.getEmployees().then((res) => {
            this.setState({ employees: res.data});
        });
    }

    render() {
        return (
            <div>
                <table className="table">
                    <thead>
                        <tr>
                            <th scope="col">ID</th>
                            <th scope="col">Name</th>
                            <th scope="col">Address</th>
                            <th scope="col">Salary</th>
                            <th scope="col">Action</th>
                        </tr>
                    </thead>
                    <tbody>
                                {
```

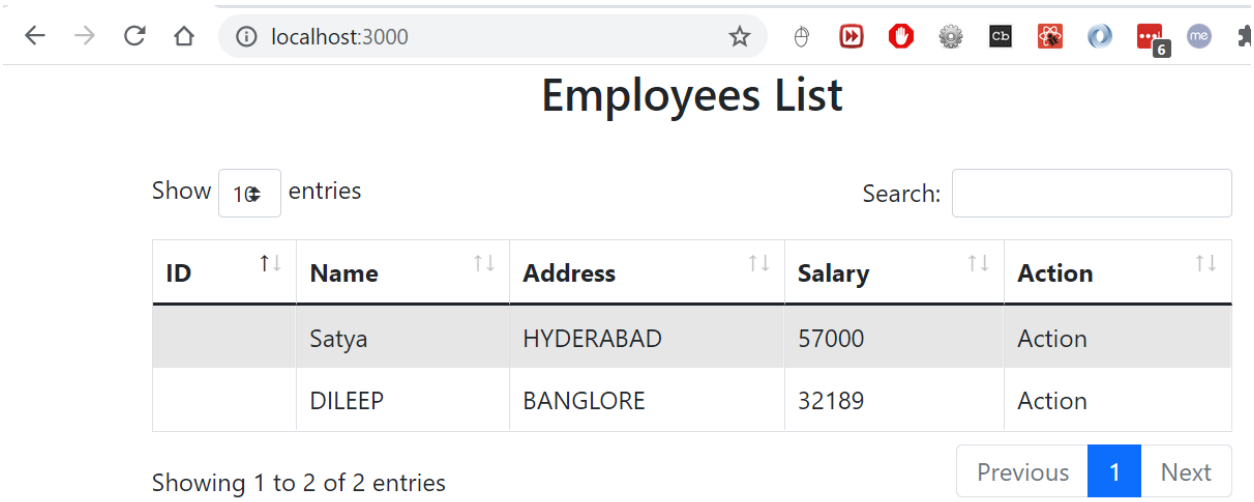
```
                                    this.state.employees.map(
                                        employee =>
                                        <tr key = {employee.id}>

                                            <td> {employee.name} </td>
                                            <td> {employee.address}</td>
                                            <td> {employee.salary}</td>
                                            <td>
                                                Action
                                            </td>
                                        </tr>
                                    )
                                }
                            </tbody>
                        </table>
                    </div>
            );
        }
}

export default ListAllEmployeesComponent;
```

employee-react-app\src\services**EmployeeService.js**

```
import axios from 'axios';
const EMPLOYEE_API_BASE_URL = "http://localhost:8080/api/v1/all";
class EmployeeService {
    getEmployees() {
        return axios.get(EMPLOYEE_API_BASE_URL);
    }
}
export default new EmployeeService()
```

Now if we open Root URL, it will display the Table Data.

http://localhost:3000



3. Header & Footer

employee-react-app\src\component**HeaderComponent.jsx**

```
import React, { Component } from 'react';

class HeaderComponent extends Component {
    render() {
        return (
            <div>
                <nav class="navbar navbar-dark bg-dark" >

                <div className="container">
                    <h3 style={{'color':'white'}}>EmployeeServices™ </h3>
                </div>

                </nav>

            </div>
        );
    }
}

export default HeaderComponent;
```

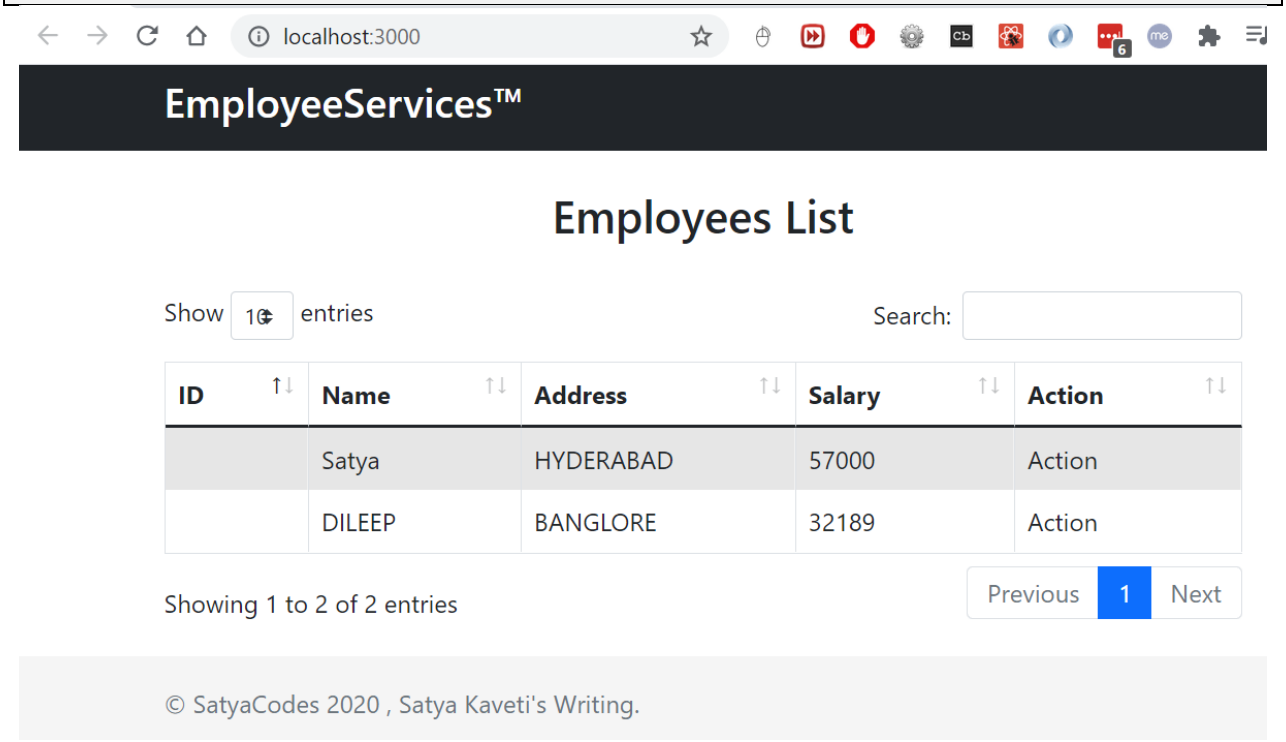
employee-react-app\src\component**FooterComponent.jsx**

```
import React, { Component } from 'react';

class FooterComponent extends Component {
    render() {
        return (
            <div>
                <footer className="footer">
                    <div className="container">
                        <span className="text-muted">
                            © SatyaCodes 2020 , Satya Kaveti's Writing.
                         </span>
                    </div>
                </footer>
            </div>
        );
    }
}

export default FooterComponent;
```

employee-react-app\src**App.js**

```
import './App.css';
import HeaderComponent from './component/HeaderComponent';
import FooterComponent from './component/FooterComponent';
import ListAllEmployeesComponent from './component/ListAllEmployeesComponent';

function App() {
  return (

    <div>
```

```
      <HeaderComponent/>
    <div className="container">
        <ListAllEmployeesComponent/>
    </div>
    <FooterComponent/>
    </div>
  );
}

export default App;
```



Routing

Install **react-router-dom**

```
npm install react-router-dom --save
```

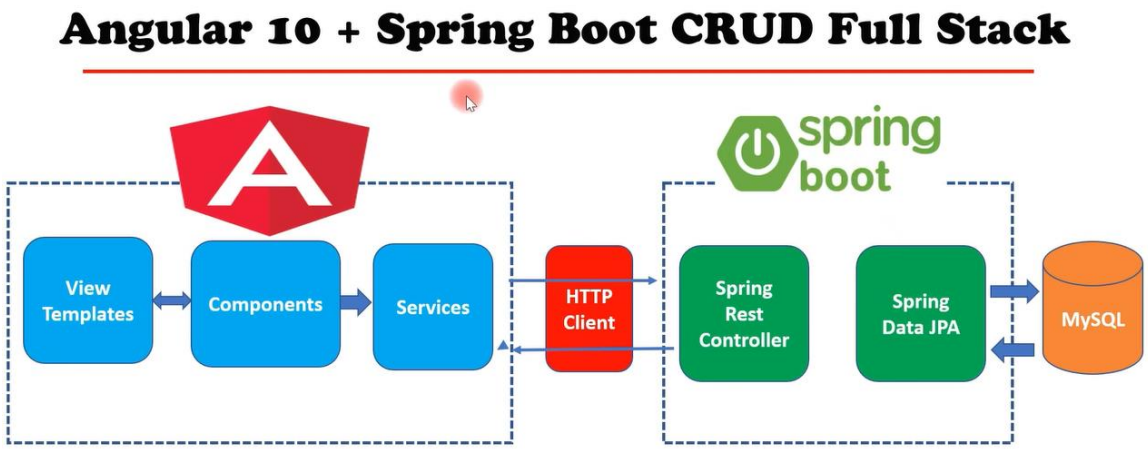
```
import './App.css';
import HeaderComponent from './component/HeaderComponent';
import HomeComponent from './component/HomeComponent';
import FooterComponent from './component/FooterComponent';
import ListAllEmployeesComponent from './component/ListAllEmployeesComponent';
import { BrowserRouter, Route, Switch } from 'react-router-dom';
import Employee from './component/Employee';
function App() {
  return (

    <div>
```

```
      <HeaderComponent />

      <BrowserRouter>
        <div className="container">
          <Switch>
               <Route exact path="/" component={HomeComponent}></Route>
               <Route exact path="/one" component={Employee}></Route>
               <Route exact path="/all" component={ListAllEmployeesComponent}></Route>
               <Route component={Error} />
          </Switch>
        </div>
      </BrowserRouter>

      <FooterComponent />
    </div>
  );
}
export default App;
```

Complete Code. Github

https://github.com/smlcodes/Books-Sync-Gitlab/tree/main/Codes/employee-react-app

#2 - Angular + SpringBoot



We are using above EmployeeServices as SpringBoot rest API, we create UI using Angular to display UI.

Create Angular Project

You use the Angular CLI to create projects, generate application and library code, and perform a variety of ongoing development tasks such as testing, bundling, and deployment.

```
npm install -g @angular/cli
```

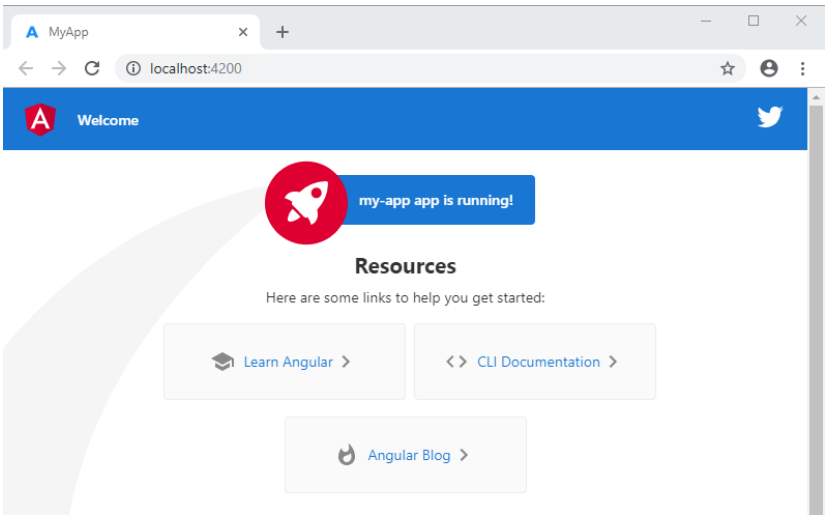
To create a new workspace and initial starter app.

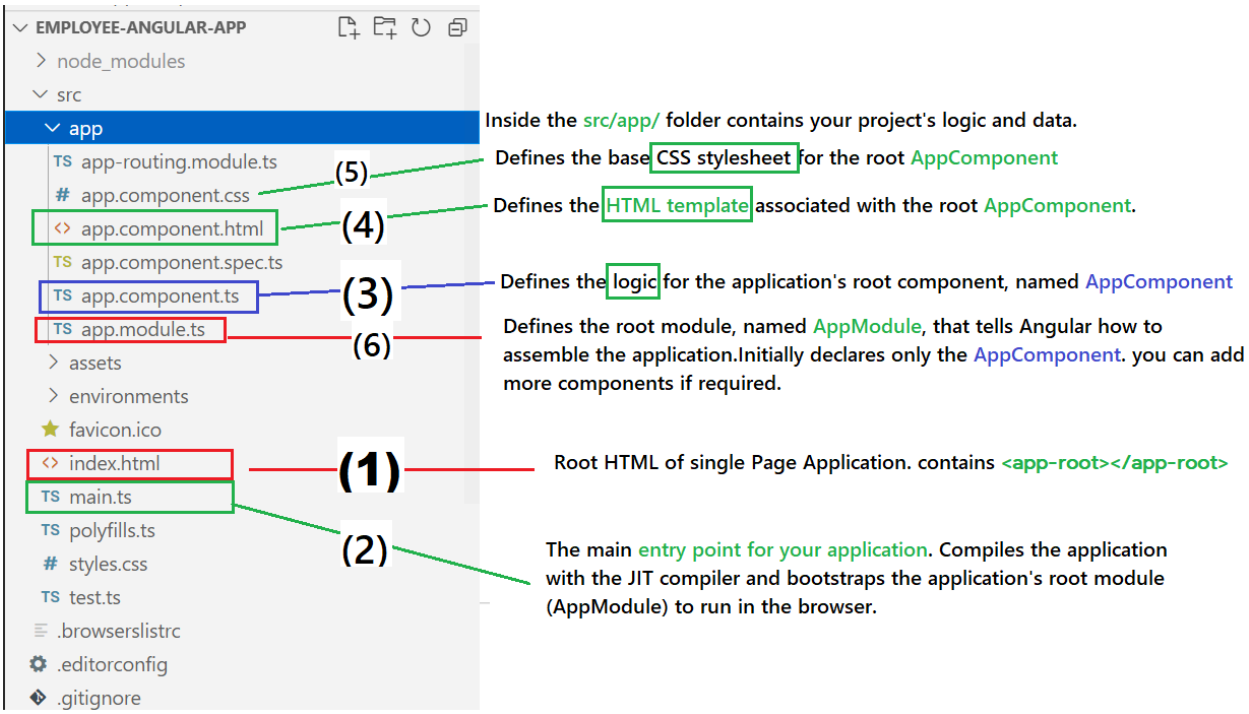
```
ng new employee-angular-app
```

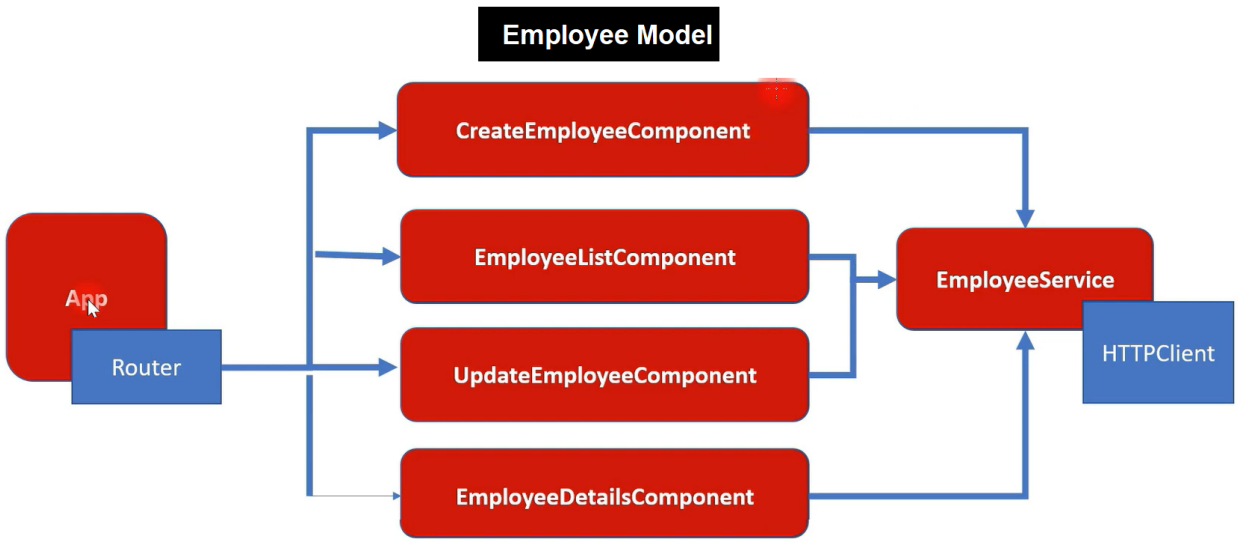
Run the application

```
ng serve --open
```

The `--open` option automatically opens your browser to `http://localhost:4200/`. If your installation and setup was successful, you should see a page similar to the following.







for generate any thing we need to use `ng g c/s`

- **g** -is for Generate
- **C** – is for Component
- **S**– is for Service

EmployeeModel - Model class to hold Employee Data

```
C:\Git\books\Codes\employee-angular-app>ng g class employee
CREATE src/app/employee.spec.ts (162 bytes)
CREATE src/app/employee.ts (26 bytes)
```

Create Employee Model class with Employee Table Properties

```
export class Employee {

    id!: number;
    name!: string;
    address!: string;
    salary!: number;

}
```

EmployeeListComponent - To Display all Employee data

```
C:\Git\books\Codes\employee-angular-app>ng g c EmployeeList
CREATE src/app/employee-list/employee-list.component.html (To display UI)
CREATE src/app/employee-list/employee-list.component.spec.ts
CREATE src/app/employee-list/employee-list.component.ts (Logic)
CREATE src/app/employee-list/employee-list.component.css (Stylesheet)
UPDATE src/app/app.module.ts (newly created component is added & imported)
```

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { EmployeeListComponent } from './employee-list/employee-list.component';

@NgModule({
  declarations: [
    AppComponent,
    EmployeeListComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

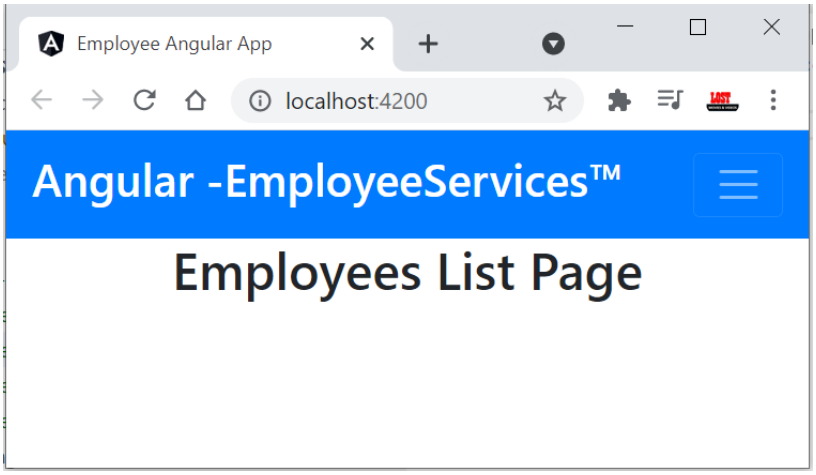
Above are automatically added after generation of component.

```
employee-list.component.html
<h1>
    Employees List Page
</h1>
```

```
//employee-list.component.ts (Default Generated Class)

import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-employee-list',
  templateUrl: './employee-list.component.html',
  styleUrls: ['./employee-list.component.css']
})
export class EmployeeListComponent implements OnInit {

  constructor() { }

  ngOnInit(): void {
  }

}
```

If we place above app-employee-list in main html(app.component.html) it will display data generated by employeelist component.

```
app.component.html
<div>
        …. Old code ….
 </div>

<app-employee-list></app-employee-list>
```



Change code to display sample Employee data

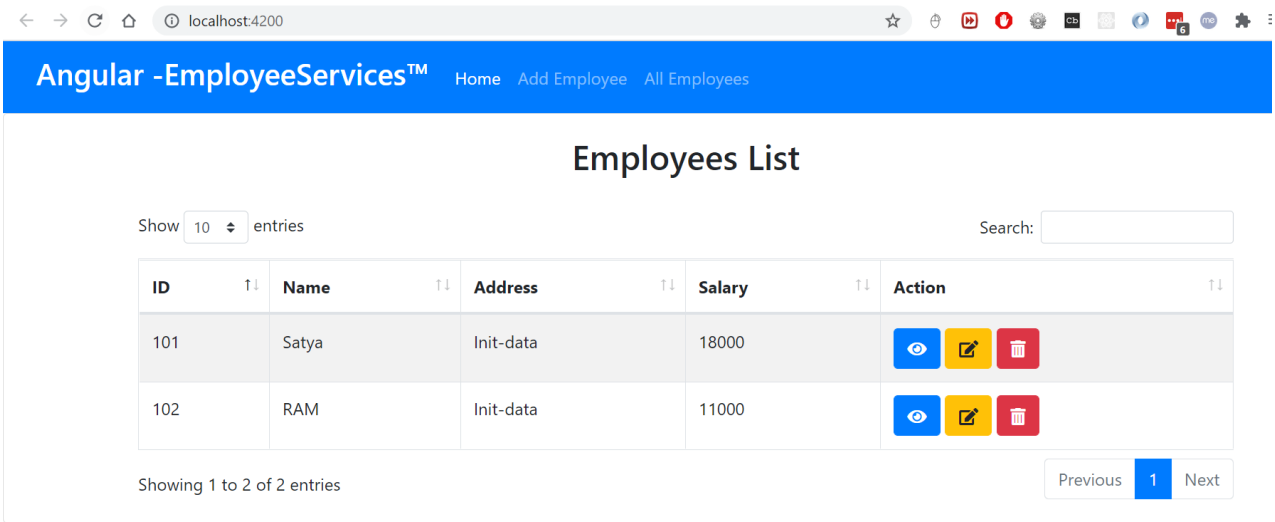
```
employee-list.component.ts
import { Component, OnInit } from '@angular/core';
import { Employee } from '../employee';

@Component({
  selector: 'app-employee-list',
  templateUrl: './employee-list.component.html',
  styleUrls: ['./employee-list.component.css'],
})
```

```
export class EmployeeListComponent implements OnInit {
  //List of emplyee objects created, initilaized to EMpty.
  employeeList!: Employee[];
  constructor() {}
  //Samle Employee Data.
  ngOnInit(): void {
    this.employeeList = [
      {
        id: 101,
        name: 'Satya',
        address: 'Init-data',
        salary: 18000,
      },
      {
        id: 102,
        name: 'RAM',
        address: 'Init-data',
        salary: 11000,
      },
    ];
  }
}
```

employee-list.component.html

```
<div class="card">
  <div class="card-body">
    <div class="col-10 mx-auto">
      <h2 class="text-center">Employees List</h2>
      <!-- <p> {{employeeList[0].name}}</p> -->
    </div>
    <br />
    <div class="row">
      <div class="col-10 mx-auto">
        <table id="example" class="table table-striped table-bordered" style="width: 100%">
          <thead>
            <tr>
              <th scope="col">ID</th>
              <th scope="col">Name</th>
              <th scope="col">Address</th>
              <th scope="col">Salary</th>
              <th scope="col">Action</th>
            </tr>
          </thead>
          <tbody>

            <tr *ngFor="let employee of employeeList">
              <td> {{employee.id}} </td>
              <td> {{employee.name}} </td>
              <td> {{employee.address}} </td>
              <td> {{employee.salary}} </td>
              <td>
                <button class="btn btn-primary"><i class="fas fa-
eye"></i> </button> <span> </span>
                <button class="btn btn-warning"><i class="fas fa-
edit"></i> </button> <span> </span>
                <button class="btn btn-danger"><i class="fas fa-trash-alt"></i> </button>

              </td>
            </tr>

          </tbody>
```

```
        </table>
      </div>
    </div>
  </div>
</div>
```



EmployeeService – To communicate with SpringBoot Services

```
C:\Git\books\Codes\employee-angular-app>ng g s employee
CREATE src/app/employee.service.spec.ts (367 bytes)
CREATE src/app/employee.service.ts (137 bytes)
```

```
//employee.service.ts (Default generated code)
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class EmployeeService {

  constructor() { }
}
```

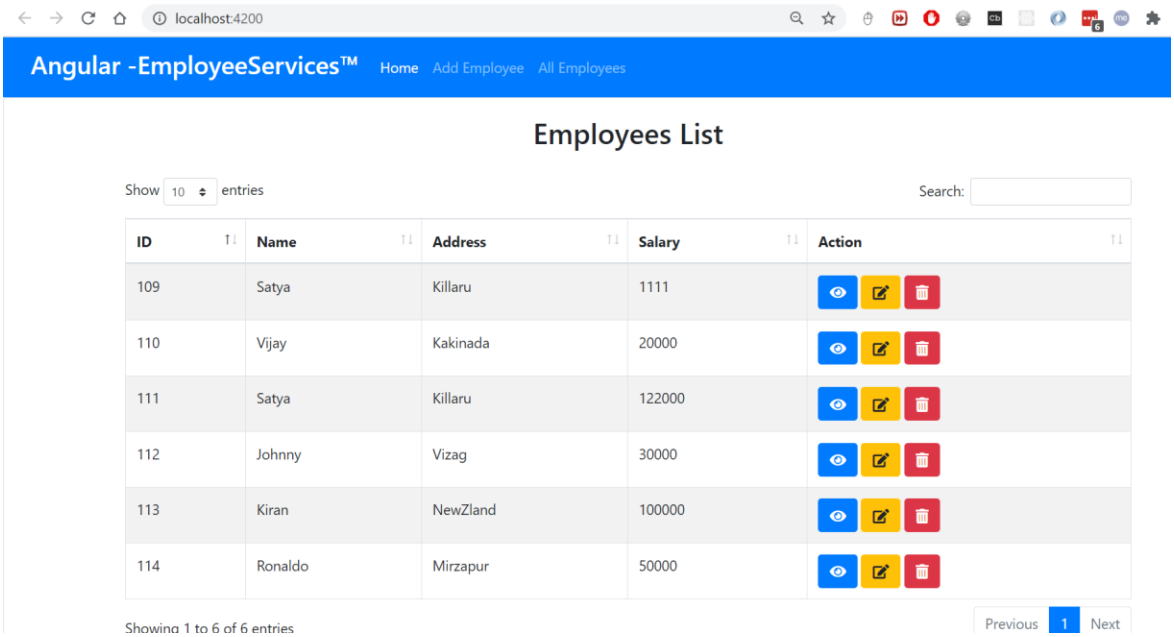
We need to use **`HttpClientModule`** to call Services. We need import that module in **`app.module.ts`**

```
//employee.service.ts (CURD Operations Rest Call)
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';
import { Employee } from './employee';

@Injectable({
  providedIn: 'root'
```

```
})
export class EmployeeService {

  private baseURL = "http://localhost:8080/api/v1";

  constructor(private httpClient: HttpClient) { }

  getEmployeesList(): Observable<Employee[]>{
    return this.httpClient.get<Employee[]>(`${this.baseURL}/all`);
  }

  createEmployee(employee: Employee): Observable<Object>{
    return this.httpClient.post(`${this.baseURL}/addEmployee`, employee);
  }

  getEmployeeById(id: number): Observable<Employee>{
    return this.httpClient.get<Employee>(`${this.baseURL}/getEmployee/${id}`);
  }

  updateEmployee(id: number, employee: Employee): Observable<Object>{
    return this.httpClient.put(`${this.baseURL}/updateEmployee/${id}`, employee);
  }

  deleteEmployee(id: number): Observable<Object>{
    return this.httpClient.delete(`${this.baseURL}/deleteEmployee/${id}`);
  }
}
```

Now call getEmployeesList() method in EmployeeListComponent to get employees data.

```
// employee-list.component.ts
import { Component, OnInit } from '@angular/core';
import { Employee } from '../employee';

//1.Imported EmployeeService
import { EmployeeService } from '../employee.service';

@Component({
  selector: 'app-employee-list',
  templateUrl: './employee-list.component.html',
  styleUrls: ['./employee-list.component.css'],
})
export class EmployeeListComponent implements OnInit {
  //List of emplyee objects created, initilaized to EMpty.
  employeeList!: Employee[];

  //2. Created EmployeeService Object to call methods
  constructor(private employeeService: EmployeeService) {}

  //3.at the time of initilaizing calling getEmployees
  ngOnInit(): void {
    this.getEmployees();
  }

  //4.Finally, employeeList object contains all empdata coming from SpringBoot
  private getEmployees() {
    this.employeeService.getEmployeesList().subscribe((data) => {
      this.employeeList = data;
    });
  }
}
```



Angular Routing

app-routing.module.ts (Default generated code)

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

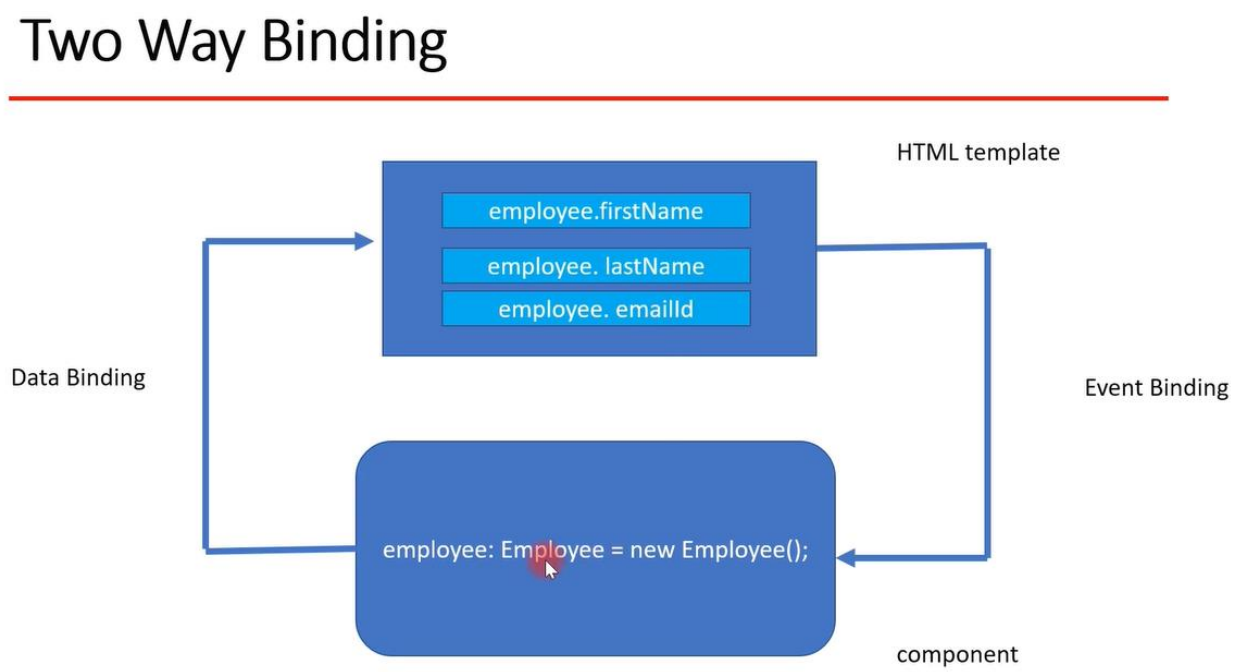
we need to configure **path** for each **component** like below

```
import { EmployeeListComponent } from './employee-list/employee-list.component';

const routes: Routes = [
  {path: 'all', component: EmployeeListComponent}
];
```

Next, in **app.component.html** – we need to place **<route-outlet>** to display the content based on **path**

```
<!--This is used directly to display all Empoyees
  <app-employee-list></app-employee-list> -->

<!-- Based on Route it will display content -->
<router-outlet></router-outlet>
```

Similarly, we need to create Components for ADD, UPADATE, DELETE employee




```
C:\Git\books\Codes\employee-angular-app>ng g c add-employee
CREATE src/app/add-employee/add-employee.component.html
CREATE src/app/add-employee/add-employee.component.ts
```

Complete Code. Github

https://github.com/smlcodes/Books-Sync-Gitlab/tree/main/Codes/employee-angular-app

Ref.

https://github.com/RameshMF/Angular-10-Spring-Boot-CRUD-Full-Stack-App

https://www.youtube.com/watch?v=tLBX9fq813c&list=PLGRDMO4rOGcNzi3CpBWsCdQSzbjdWWy-f

SpringBoot + ThymeLeaf

https://www.youtube.com/watch?v=U7WDfoiodOg&list=PLGRDMO4rOGcM7B0TsM_-2efML7LAOriWV

Dockerize Spring MVC Application

https://mkyong.com/docker/docker-spring-boot-examples/

https://medium.com/@asce4s/dockerize-spring-mvc-application-a9ffbd11eadb

Deploying Spring Boot Applications to Heroku

https://devcenter.heroku.com/articles/deploying-spring-boot-apps-to-heroku

SpringBoot – Micro Services

https://www.sivalabs.in/2018/03/microservices-using-springboot-spring-cloud-part-1-overview/

https://github.com/sivaprasadreddy/spring-boot-microservices-series

Examples

https://github.com/spring-petclinic/spring-petclinic-microservices

https://github.com/kbastani/spring-cloud-microservice-example

MyTrails CTMS Sample project

Phoenix CTMS

https://github.com/phoenixctms/ctsms

- **PRS** (Patient Recruitment System)
- **CTMS** (Clinical Trial Management System)
- **CDMS** (Clinical Data Management System)

LibreClinica

https://github.com/reliatec-gmbh/LibreClinica